Topological Data Analysis를 활용한 음악 구조 분석 및 위상 구조 보존 기반 AI 작곡 파이프라인

저자: 김민주
지도: 정재훈 (KIAS 초학제 독립연구단)
작성일: 2026.04.16
키워드: Topological Data Analysis, Persistent Homology, Tonnetz, Music Generation, Vietoris-Rips Complex, Jensen-Shannon Divergence

---

초록 (Abstract)

본 연구는 사카모토 류이치의 2009년 앨범 out of noise 수록곡 "hibari"를 대상으로, 음악의 구조를 **위상수학적으로 분석**하고 그 위상 구조를 **보존하면서 새로운 음악을 생성**하는 파이프라인을 제안한다. 전체 과정은 네 단계로 구성된다. (1) MIDI 전처리: 두 악기를 분리하고 8분음표 단위로 양자화. (2) Persistent Homology: 네 가지 거리 함수(frequency, Tonnetz, voice leading, DFT)로 note 간 거리 행렬을 구성한 뒤 H_1 cycle을 추출. (3) 중첩행렬 구축: cycle의 시간별 활성화를 이진 또는 연속값 행렬로 기록. (4) 음악 생성: 중첩행렬을 seed로 하여 확률적 샘플링 기반의 Algorithm 1과 FC / LSTM / Transformer 신경망 기반의 Algorithm 2 두 방식으로 음악 생성.

$N=20$회 통계적 반복을 통한 정량 검증에서 (최소 onset 간격 $\mathrm{gap_min}=3$, 1.5박 간격 제약 적용), **Algorithm 1**(확률적 샘플링) 기반으로 DFT 거리 함수가 네 거리 함수 중 최우수로 확인되었다 — frequency 대비 9.3% 개선 ($0.0319 \to 0.0289$), Tonnetz 대비 47.8% 개선 ($0.0554 \to 0.0289$). 가중치 최적화 ($w_o=0.3$, $w_d=1.0$, N=10 grid search) 후 DFT 이진 중첩행렬 기반 Algorithm 1은 JS divergence 0.0185 ± 0.0022를 달성한다. DFT 조건에서는 이진 OM이 연속값 OM보다 우수하였는데 (0.0185 vs 0.0231), 이는 DFT 거리가 스펙트럼 구조를 이미 정밀하게 포착하여 이진 표현만으로도 cycle 활성 신호가 충분히 구별되기 때문으로 해석된다. **Algorithm 2**(DL 기반 생성)에서는 Transformer 모델이 DFT 이진 OM 입력 기반으로 JS divergence 0.0028 ± 0.0008를 달성하여 최우수를 기록하였다 ($N=5$). Transformer의 주의(attention) 메커니즘이 DFT 기반 cycle 구조를 더 효과적으로 학습함을 시사하며, LSTM은 동일 조건에서 0.240 ± 0.032로 크게 열화하였다.

본 연구의 intra / inter / simul 세 갈래 가중치 분리 설계는 hibari의 두 악기 구조 — inst 1은 쉼 없이 연속 연주, inst 2는 모듈마다 규칙적 쉼을 두며 겹쳐 배치 — 를 수학적 구조에 반영한 것이며, 두 악기의 활성/쉼 패턴 관측 (inst 1 쉼 0개, inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다.

---

1. 서론 — 연구 배경과 동기

1.1 연구 질문

음악은 시간 위에 흐르는 소리들의 집합이지만, 그 "구조"는 단순한 시간 순서만으로 포착되지 않는다. 같은 모티브가 여러 번 반복되고, 서로 다른 선율이 같은 화성 기반 위에서 엮이며, 전혀 관계없어 보이는 두 음이 같은 조성 체계 안에서 등가적 역할을 한다. 이러한 층위의 구조를 수학적으로 포착하려면 "어떤 두 대상이 같다(혹은 가깝다)"를 정의하는 **거리 함수**와, 그로부터 파생되는 **위상 구조**를 다루는 도구가 필요하다.

본 연구는 다음의 세 가지 질문에서 출발한다.

1. **위상 구조를 "보존한 채" 새로운 음악을 생성할 수 있는가?** 보존의 기준은 무엇이며, 보존 정도를 어떻게 정량적으로 측정하는가?
2. **거리 함수의 선택이 실제로 생성 품질에 유의미한 영향을 주는가?** 단순 빈도 기반 거리 대신 음악 이론적 거리(Tonnetz, voice leading, DFT)를 사용하면 얼마나 나은가?
3. **위상 구조를 보존한 음악이 실제로 아름답게 들리는가?** 수학적으로 유사한 위상 구조를 가지도록 생성된 음악이 청각적으로도 원곡의 미학적 인상을 전달하는가? 본 보고서 말미에 첨부된 QR코드를 통해 생성된 음악을 직접 감상할 수 있다.

1.2 연구 대상 — 왜 hibari인가

본 연구의 대상곡은 사카모토 류이치의 out of noise (2009) 수록곡 "hibari" 이다. 이 곡을 선택한 이유는 다음과 같다.

선행연구의 확장에 적합. 단선율의 국악에 TDA를 적용한 정재훈 교수의 선행연구(정재훈 외, 2024)를 화성음악으로 확장함에 있어, hibari는 복잡성을 내포하면서도 규칙적인 모듈 구조로 일정한 패턴이 있어 모델링이 용이하였다.

미학적 특수성. out of noise 앨범은 "소음과 음악의 경계"를 탐구하는 실험적 작업이며, hibari는 전통적 선율 진행이 아니라 음들의 공간적 배치에 가까운 방식으로 구성된다. 이 특성은 본 연구의 실험 결과 (§4.3)에서 DL 모델 선택과 직접적으로 공명한다.

---

2. 수학적 배경

본 절에서는 본 연구의 파이프라인을 이해하기 위해 필요한 수학적 도구들을 정의하고, 각 도구가 음악 구조 분석에서 어떻게 사용되는지를 서술한다.

2.1 Vietoris-Rips Complex

정의 2.1. 거리 공간 (X, d)와 양의 실수 $\varepsilon > 0$이 주어졌을 때, **Vietoris-Rips complex** $\mathrm{VR}_\varepsilon(X)$는 다음과 같이 정의되는 복합체(simplicial complex)이다:

$$\mathrm{VR}_\varepsilon(X) = \left\{\sigma \subseteq X \mid \forall x_i, x_j \in \sigma,\ d(x_i, x_j) \leq \varepsilon\right\}$$

즉, 점 집합 X의 부분집합 σ에 속한 **모든 점 쌍 사이의 거리가** ε **이하**이면 σ를 심플렉스(simplex)로 포함시킨다.

구성 요소:

0-simplex (vertex): 각 점 $x_i \in X$. 단일 점은 거리 조건이 없으므로 어떤 ε에서도 포함된다.

1-simplex (edge): $d(x_i, x_j) \leq \varepsilon$인 두 점의 쌍 $\{x_i, x_j\}$

2-simplex (triangle): 세 점이 서로 모두 ε 이내인 부분집합

Filtration 구조와 포함관계. ε 값을 0부터 연속적으로 키우면, 점 집합 X 자체는 변하지 않은 채 **새로운 심플렉스만 점차 추가된다.** $\varepsilon = 0$일 때 $\mathrm{VR}_0(X)$는 각 점만을 0-simplex로 포함하는 이산적인 점 집합(discrete set)이다 — 아직 어떤 edge도 없으므로 이것은 X 그 자체와 같다. ε이 커지면서 두 점 사이 거리가 ε 임계를 처음 넘는 순간에 1-simplex(edge)가 추가되고, 세 점이 모두 ε 이내가 되면 2-simplex(삼각형)가 추가된다. 즉 $\varepsilon_1 < \varepsilon_2$이면 $\mathrm{VR}_{\varepsilon_1}(X)$의 모든 심플렉스가 $\mathrm{VR}_{\varepsilon_2}(X)$에도 그대로 들어 있다. 따라서 다음의 포함관계는 항상 성립한다:

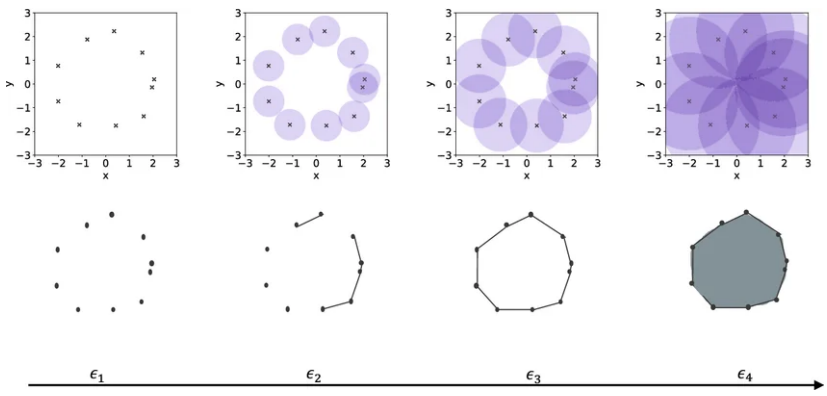
$$\mathrm{VR}_0(X) \subseteq \mathrm{VR}_{\varepsilon_1}(X) \subseteq \mathrm{VR}_{\varepsilon_2}(X) \subseteq \cdots \subseteq \mathrm{VR}_{\varepsilon_n}(X)$$

표기 편의를 위해 $K_i := \mathrm{VR}_{\varepsilon_i}(X)$로 두면:

$$K_0 \subseteq K_1 \subseteq K_2 \subseteq \cdots \subseteq K_n$$

이를 **filtration**이라 부르며, 변화가 일어나는 임계값 ε_i들이 곧 심플렉스의 birth/death 시점이 된다.

본 연구에서의 사용: $X = \{n_1, n_2, \ldots, n_{23}\}$은 hibari에 등장하는 23개의 고유 note이며, $d(n_i, n_j)$는 두 note 간 거리이다.



2.2 Simplicial Homology

정의 2.2. Simplex complex K에 대해 n차 호몰로지 군(homology group) $H_n(K)$는 K 안에 존재하는 n차원 "구멍"의 대수적 표현이다. 직관적으로:

$H_0(K)$: 연결 성분(connected components)의 수

$H_1(K)$: 1차원 cycle의 수 (닫힌 고리 모양으로 둘러싸인 영역)

$H_2(K)$: 2차원 빈 공간(void)의 수 (3차원 공동을 둘러싼 표면)

$H_n(K)$는 아벨 군이며, $\mathrm{rank}(H_n(K))$ = **Betti number** β_n은 서로 독립적인 n차원 구멍의 개수를 나타낸다.

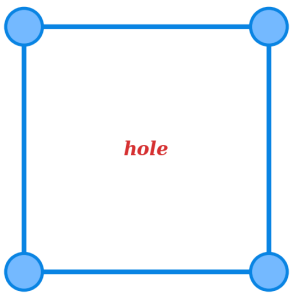
Boundary 연산자와 호몰로지 — 선형대수적 계산 예시. H_n이 어떻게 계산되는지를 간단한 예시로 보인다. 점 4개 $\{a, b, c, d\}$와 edge $\{ab, bc, cd, da, ac\}$, 삼각형 $\{abc\}$로 이루어진 복합체 K를 생각하자.

boundary 연산자 ∂_1 (edge → vertex)과 ∂_2 (삼각형 → edge)를 $\mathbb{F}_2$ (mod 2) 위에서 행렬로 표현하면:

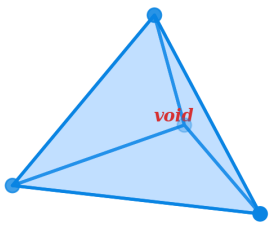
```
∂₁ (edge → vertex):          ∂₂ (triangle → edge):
      ab bc cd da ac                 abc
a [   1  0  0  1  1 ]         ab [   1 ]
b [   1  1  0  0  1 ]         bc [   1 ]
c [   0  1  1  0  0 ]         cd [   0 ]
d [   0  0  1  1  0 ]         da [   0 ]
                              ac [   1 ]
```

$H_1(K) = \ker(\partial_1)/\mathrm{im}(\partial_2)$이다. $\ker(\partial_1)$은 "닫힌 edge 체인들"의 공간이고, $\mathrm{im}(\partial_2)$는 "삼각형의 경계인 edge 체인들"의 공간이다. 이 예시에서 $\ker(\partial_1)$의 차원은 2 (두 개의 독립적인 닫힌 고리: $ab + bc + ca$와 $ac + cd + da$), $\mathrm{im}(\partial_2)$의 차원은 1 (abc의 경계 $= ab + bc + ca$). 따라서 $\beta_1 = 2 - 1 = 1$, 즉 독립적인 cycle이 1개이다. 직관적으로, 삼각형 abc가 한 cycle을 "채워서" 없앴고, 남은 사각형 $a - c - d - a$에 해당하는 cycle 1개가 살아남는다.

(a) 1D cycle: generator of H_1
4 points forming a square boundary,
no 2-simplex inside



(b) 2D void: generator of H_2
4 points forming a tetrahedron boundary,
no 3-simplex inside



본 연구에서의 사용: 본 연구는 주로 H_1 (1차 호몰로지)을 다룬다. 이는 음악 네트워크에서 서로 가까운 note들이 만드는 닫힌 cycle, 즉 순환적으로 연결된 note 그룹을 포착한다. 발견된 각 cycle은 곡의 구조적 반복 단위로 해석된다.

2.3 Persistent Homology

Filtration $K_0 \subseteq K_1 \subseteq \cdots \subseteq K_n$에서, 각 단계마다 H_1의 cycle 구성이 달라진다. **Persistent homology**는 이 과정에서 각 cycle이 어느 ε_i에서 처음 나타나고(**birth**) 어느 ε_j에서 사라지는지(**death**)를 추적한다.

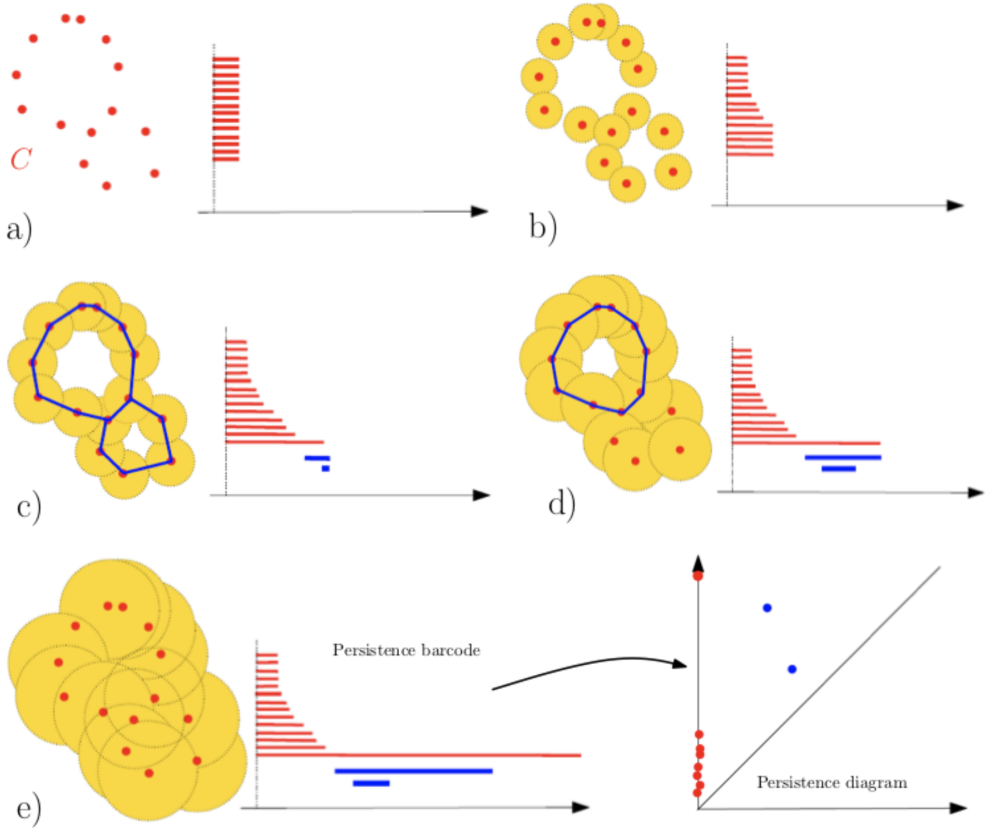
Birth와 death의 음악적 의미:

- **Birth** b: 거리 임계값 ε가 충분히 커져서 새로운 cycle이 형성되는 순간. 음악적으로는 "이 거리 척도에서 처음으로 닫힌 반복 구조가 발견되는 시점".
- **Death** d: 더 큰 ε에서 그 cycle이 다른 cycle들의 합(정확히는 boundary)으로 표현될 수 있게 되어, 호몰로지 군 안에서 더 이상 독립적인 generator가 아니게 되는 순간. 음악적으로는 "거리 척도가 너무 느슨해져서 이 반복 구조가 다른 구조에 흡수되는 시점".

각 cycle은 (b, d) 쌍과, 그 cycle을 구성하는 vertex/edge 집합인 **cycle representative**가 함께 기록된다. 본 연구에서는 birth-death 쌍은 cycle의 "수명"을 측정하는 데, cycle representative는 어떤 note들이 그 cycle을 이루는지 식별하는 데 사용한다. 이 정보를 모은 것이 곡의 **위상적 지문(topological signature)**이다.

Persistence: $\mathrm{pers}(\mathrm{cycle}) = d - b$. 큰 persistence를 갖는 cycle은 다양한 거리 척도에서 살아남으므로 **위상적으로 안정한 구조**이며, 작은 persistence는 일시적이거나 노이즈에 가까운 구조이다.

본 연구에서의 사용: 거리 행렬 $D \in \mathbb{R}^{23 \times 23}$로부터 Vietoris-Rips filtration을 구성하고, 각 rate parameter r (가중치 비율, 후술)에서의 H_1 persistence를 계산한다. 발견된 모든 (b, d) 쌍과 cycle representative가 함께 cycle 집합을 정의하며, 이 cycle들이 다음 절의 중첩행렬 구축에 사용된다.



2.4 빈도 기반 거리와 음악적 거리 함수

빈도 기반 거리. 본 연구의 기준 거리 d_{freq}는 두 note의 인접도(adjacency)의 역수로 정의된다. 인접도 $w(n_i, n_j)$는 곡 안에서 note n_i와 n_j가 시간적으로 연달아 등장한 횟수이다:

$$w(n_i, n_j) = \#\{\, t : n_i \text{ at time } t \text{ and } n_j \text{ at time } t+1 \,\}$$

거리는 $d_{\text{freq}}(n_i, n_j) = 1/w(n_i, n_j)$로 정의되며 인접도가 0인 경우는 도달 불가능한 큰 값으로 처리한다(§2.9).

정의 2.4. Tonnetz는 pitch class 집합 $\mathbb{Z}/12\mathbb{Z}$를 평면 격자에 배치한 구조이다. 여기서 **pitch class**는 옥타브 차이를 무시한 음의 동치류(equivalence class)로, 예컨대 C4 (가운데 도), C5 (한 옥타브 위 도), C3 등은 모두 같은 pitch class "C"에 속한다.

Tonnetz의 격자 구조. pitch class $p \in \mathbb{Z}/12$를 좌표 (x, y)에 배치하되, 다음 관계를 만족시킨다:

가로 이동 (+1 in x): 완전5도 (perfect fifth, +7 semitones)

대각선 이동 (+1 in y): 장3도 (major third, +4 semitones)

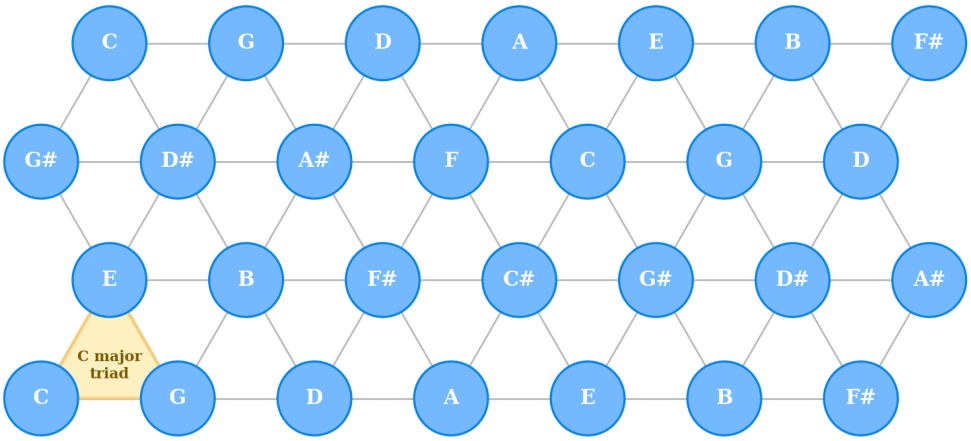


그림 2.4. Tonnetz 격자 구조. 가로 방향은 완전5도(C→G→D→A→E⋯), 대각선 방향은 장3도(C→E→G#⋯)와 단3도(C→A→F#⋯)로 이동한다. 삼각형 하나는 하나의 장3화음(major triad) 또는 단3화음(minor triad)에 대응된다.

(1) Tonnetz 거리. 두 pitch class p_1, p_2 사이의 Tonnetz 거리 $d_T(p_1, p_2)$는 격자 위 최단 경로 길이로 정의된다:

$$d_T(p_1, p_2) = \min\{\, |x_1 - x_2| + |y_1 - y_2| \mid (x_i, y_i) \text{ represents } p_i \,\}$$

(2) Voice leading distance (Tymoczko, 2008): 두 pitch class 사이를 이동하기 위해 거쳐야 하는 반음의 개수와 같다.

$$d_V(p_1, p_2) = |p_1 - p_2|$$

(3) DFT distance (Tymoczko, 2008): 각 pitch class를 12차원 벡터로 표현한 뒤, 이산 푸리에 변환(DFT)으로 다른 공간으로 옮겨서 비교한다.

복합 거리(Hybrid distance). 본 연구는 빈도 기반 거리 d_{freq}와 음악적 거리 d_{music} (Tonnetz, Voice leading, DFT 중 하나)을 선형 결합한다:

$$d_{\text{hybrid}}(n_i, n_j) = \alpha \cdot d_{\text{freq}}(n_i, n_j) + (1 - \alpha) \cdot d_{\text{music}}(n_i, n_j)$$

본 연구에서의 사용: 거리 함수의 선택은 발견되는 cycle 구조에 직접적으로 영향을 미친다. 빈도 기반 거리만 사용하면 곡의 통계적 특성만 반영되어 화성적·선율적 의미가 있는 구조를 포착하지 못한다.

2.5 활성화 행렬과 중첩행렬

본 연구에서는 곡의 시간축 위에서 cycle 구조가 어떻게 전개되는지를 두 단계의 행렬로 표현한다. 첫 단계는 **활성화 행렬(activation matrix)**, 두 번째 단계는 그것을 가공한 **중첩행렬(overlap matrix, OM)**이다.

정의 2.5 (활성화 행렬). 음악의 시간축 길이를 T, 발견된 cycle의 수를 C라 하자. 활성화 행렬 $A \in \{0,1\}^{T \times C}$는 raw 활성 정보를 담는다:

$$A[t,c] = \mathbb{1}[\,\exists\, n \in V(c) \text{ such that } n \text{ is played at time } t\,]$$

여기서 $V(c)$는 cycle c의 note 집합이다. 활성화 행렬은 산발적인 단일 시점 활성화까지 모두 포함하므로 노이즈가 많다.

정의 2.6 (OM). OM $O \in \{0,1\}^{T \times C}$는 활성화 행렬에서 **연속적이고 충분히 긴 활성 구간만 남긴 것**이다.

$$O[t,c] = \mathbb{1}[\,t \in R(c)\,], \qquad R(c) = \bigcup_i [s_i,\ s_i + L_i]$$

여기서 $R(c)$는 cycle c의 "지속 활성 구간(sustained intervals)"의 합집합이며, 각 구간 $[s_i, s_i + L_i]$는 활성화 행렬 $A[\cdot,c]$에서 길이가 임계값 scale_c 이상인 연속 1의 구간이다.

활성화 행렬과 OM의 차이.

- $A[t,c]$: 시점 t에 cycle c의 note가 단 한 번이라도 울리면 1. **순간적 활성을 모두 잡음.**
- $O[t,c]$: cycle c의 활성이 일정 시간 이상 **지속되는 구간**에서만 1. 산발적 노이즈 제거됨.

예를 들어 $\text{scale}_c = 3$일 때 (3 시점 이상 지속된 활성만 인정), 아래와 같다.

```
     :  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
A[·,c]: 0  1  1  0  1  1  1  1  0  0  1  0  1  1  1
O[·,c]: 0  0  0  0  1  1  1  1  0  0  0  0  1  1  1
```

본 연구에서 OM을 음악 생성의 seed로 사용하는 이유는, 잠시 스쳐가는 활성보다 일정 시간 유지되는 cycle만이 곡의 구조적 단위로 의미 있다고 보기 때문이다.

연속값 확장. 본 연구에서는 이진 OM 외에, cycle의 활성 정도를 [0,1] 사이의 실수값으로 표현하는 연속값 버전도 도입하였다:

$$O_{\text{cont}}[t,c] = \frac{\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[\,n \text{ is played at time } t\,]}{\sum_{n \in V(c)} w(n)}$$

여기서 $V(c)$는 cycle c의 vertex 집합, $w(n) = 1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 note가 활성화되면 더 큰 가중치를 받는다.

음악적 의미: OM은 곡의 **위상적 뼈대(topological skeleton)**를 시각화한 것이다. 시간이 흐름에 따라 어떤 반복 구조가 켜지고 꺼지는지를 나타내며, 이것이 음악 생성의 seed 역할을 한다.

2.6 Jensen-Shannon Divergence — 생성 품질의 핵심 지표

JS divergence는 두 확률 분포가 얼마나 다른지를 측정하는 지표이다. 값의 범위는 $[0, \log 2]$이며, 0이면 두 분포가 동일하다.

본 연구에서 비교하는 두 가지 분포:

1. **Pitch 빈도 분포** — "어떤 음들이 얼마나 자주 쓰였는가" (시간 순서 무시)
2. **Transition 빈도 분포** — "어떤 음 다음에 어떤 음이 오는가" (시간 순서 반영)

두 지표를 함께 사용함으로써 "음을 비슷하게 쓰는가"와 "비슷한 순서로 쓰는가"를 별도로 측정할 수 있다.

2.7 Greedy Forward Selection

발견된 전체 cycle 집합 $\mathcal{C}$에서 원곡의 위상 구조를 가장 잘 보존하는 부분집합 $S \subseteq \mathcal{C}$를 선택해야 한다. 이를 위해 **greedy forward selection**을 사용한다: 보존도 함수 $f(S) = 0.5 \cdot J(S) + 0.3 \cdot C(S) + 0.2 \cdot B(S)$를 정의하고, 매 단계마다 f를 가장 크게 증가시키는 cycle을 하나씩 추가한다.

세 지표는 각각:

- **Note Pool Jaccard** $J(S)$: 선택된 cycle들이 전체 note를 얼마나 커버하는가
- **Overlap pattern correlation** $C(S)$: 시점별 활성 패턴이 원본과 얼마나 동조하는가 (Pearson 상관)
- **Betti curve similarity** $B(S)$: rate 변화에 따른 전체 위상 복잡도의 골격이 보존되는가

Note Pool Jaccard에 가장 큰 비중(0.5)을 둔 이유는 음악 생성의 직접적 입력이 cycle 구성 note이기 때문이다. 실험적으로 greedy 방법이 46개 cycle 중 15개로 90% 보존도를 달성하는 것을 확인하였다.

2.8 Multi-label Binary Cross-Entropy Loss

각 시점에서 동시에 여러 note가 활성화될 수 있으므로, 단일 클래스 예측인 categorical cross-entropy 대신 **multi-label BCE**를 사용한다. 각 note 채널마다 독립적인 binary cross-entropy를 계산하여 "note i가 활성인가?"를 개별 이진 문제로 학습한다. 모델 입력은 OM의 한 행 $O[t, :] \in \mathbb{R}^C$이고, 출력은 N차원 multi-hot vector이다.

Adaptive threshold: 추론 시 고정 임계값 0.5 대신, 원곡의 평균 ON 비율(약 15%)에 맞춰 상위 15%에 해당하는 sigmoid 출력만 활성으로 채택하는 동적 임계값을 사용한다.

2.9 음악 네트워크 구축과 가중치 분리

rate parameter r_t의 의미. r_t는 timeflow weight에서 intra weights와 inter weight의 비중을 조절한다.

- $r_t = 0$: $W = W_{\text{intra}}$만 사용. 각 악기의 선율적 흐름만 반영.
- $r_t = 1$: intra와 inter를 동등하게 결합. 선율과 상호작용을 균형 있게 반영.
- $r_t > 1$: inter의 비중이 intra보다 커짐. 악기 간 상호작용이 지배적인 구조를 탐색.

가중치 행렬의 분리 (본 연구의 핵심 설계): 본 연구는 가중치를 다음과 같이 세 가지로 분리한다:

1. **Intra weights** : 같은 악기 내에서 연속한 두 화음 간 전이 빈도. 각 악기의 **선율적 흐름**을 포착한다.

$$W_{\text{intra}} = W_{\text{intra}}^{(1)} + W_{\text{intra}}^{(2)}$$

2. **Inter weight** : 시차(lag) ℓ을 두고 악기 1의 화음과 악기 2의 화음이 동시에 출현하는 빈도이다. $\ell \in \{1, 2, 3, 4\}$로 변화시키며 다양한 시간 스케일의 **악기 간 상호작용**을 탐색한다. 가까운 시차에 더 큰 비중을 두는 **감쇄 가중치** λ_ℓ을 사용하여 합산한다:

$$W_{\text{inter}} = \sum_{\ell=1}^{4} \lambda_\ell \cdot W_{\text{inter}}^{(\ell)}, \qquad (\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (0.6,\ 0.3,\ 0.08,\ 0.02)$$

가중치는 "먼 시차의 우연한 동시 등장보다 가까운 시차의 인과적 상호작용이 음악적으로 의미 있다"는 가정을 반영한다.

3. **Simul weight** : 같은 시점에서 두 악기가 동시에 타건하는 note 조합의 빈도. **순간적 화음 구조**를 포착한다.

Timeflow weight (선율 중심 탐색):

$$W_{\text{timeflow}}(r_t) = W_{\text{intra}} + r_t \cdot W_{\text{inter}}$$

$r_t \in [0, 1.5]$를 변화시키며 위상 구조의 출현·소멸을 추적한다.

Complex weight (선율-화음 결합):

$$W_{\text{complex}}(r_c) = W_{\text{timeflow}} + r_c \cdot W_{\text{simul}}$$

$r_c \in [0, 0.5]$로 제한하여 "음악은 시간 예술이므로 화음보다 선율에 더 큰 비중을 둔다"는 가정을 반영한다.

거리 행렬: 가중치 $w(n_i, n_j) > 0$에 대해 거리는 역수로 정의된다:

$$d(n_i, n_j) = 1 \,/\, w(n_i, n_j) \;\; \text{if} \;\; w(n_i, n_j) > 0, d_\infty \;\; \text{otherwise}$$

Figure 7. 두 악기의 모듈 반복 — Inst 1은 연속, Inst 2는 모듈마다 쉼

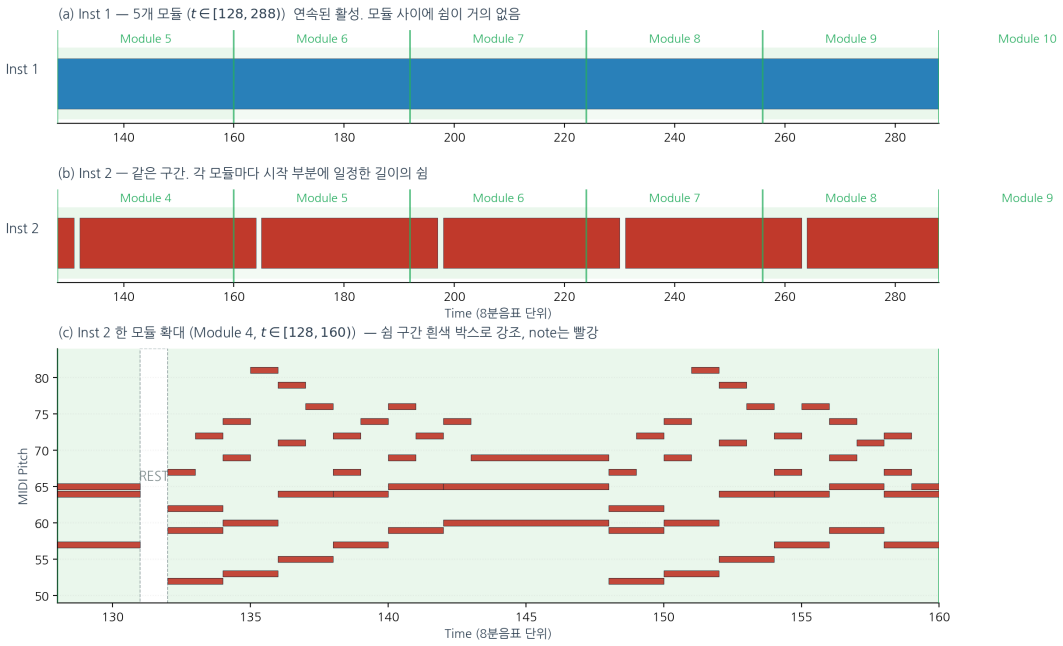


그림 2.9. hibari의 두 악기 배치 구조. inst 1 (위)은 전체 $T = 1{,}088$ 시점에서 쉬지 않고 연주하며 (쉼 0개), inst 2 (아래)는 32-timestep 모듈마다 규칙적인 쉼을 두고 얹힌다 (쉼 64개). 이 비대칭 배치가 가중치 행렬의 intra / inter / simul 분리의 근거가 된다.

2.10 확장 수학적 도구 — 거리 보존 재분배와 화성 제약

본 절은 §6의 확장 실험에서 사용되는 도구를 간략히 소개한다. 상세 수식은 해당 절에서 필요한 시점에 도입한다.

Persistence Diagram Wasserstein Distance: 두 barcode의 birth-death 점들을 최적 매칭한 이동 비용. 두 위상 구조의 유사도를 직접 비교하는 데 사용한다.

Consonance score: 시점별 동시 타건 note 쌍의 roughness(불협화도) 평균. 음악이론의 협화도 분류에 기반하여, 생성된 음악의 화성적 질을 평가한다.

Markov chain 시간 재배치: 원본 OM의 행 전이 패턴을 학습하여 새로운 시간 순서를 재샘플링하는 기법.

---

3. 두 가지 음악 생성 알고리즘

표기 정의

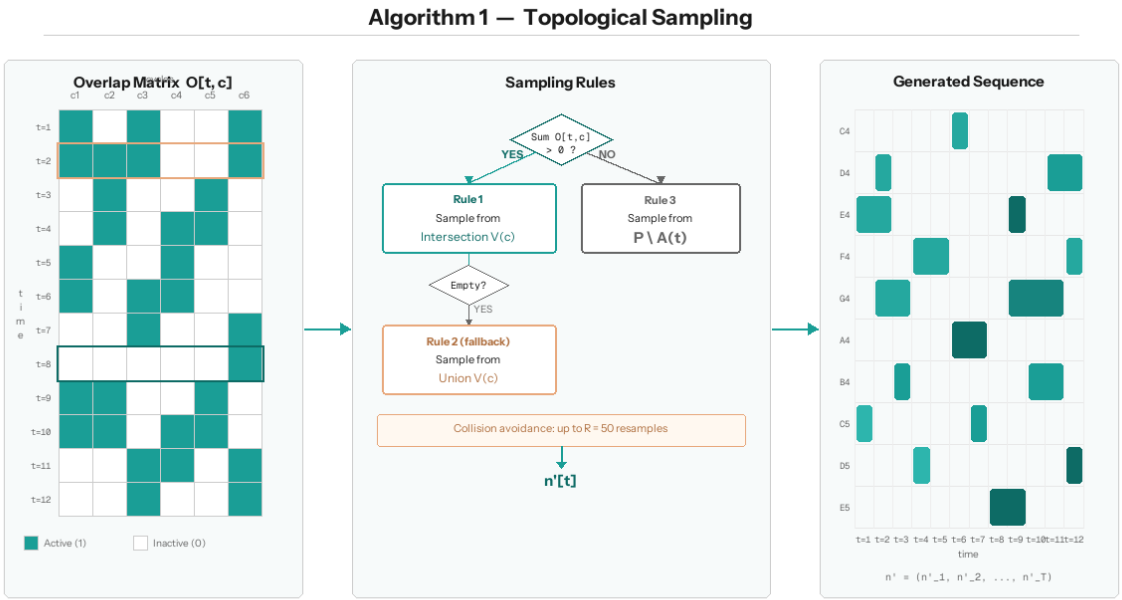
본 장에서 사용할 표기를 다음과 같이 통일한다.

기호	의미	hibari 값
T	시간축 길이 (8분음표 단위)	$1{,}088$
N	고유 note 수 (pitch-duration 쌍)	23
C	발견된 전체 cycle 수	최대 52

K	선택된 cycle subset 크기 ($K \le C$)	$\{10, 17, 46\}$
O	OM, $\{0,1\}$ 값의 $T \times K$ 행렬	—
L_t	시점 t에서 추출할 note 개수	3 또는 4
$V(c)$	cycle c의 vertex(note label) 집합	원소 수 $4 \sim 6$
R	재샘플링 최대 시도 횟수	50
B	학습 미니배치 크기	32
E	학습 epoch 수	200
H	DL 모델의 hidden dimension	128

3.1 Algorithm 1 — 확률적 샘플링 기반 음악 생성

> **참고**: Algorithm 1의 3가지 샘플링 규칙은 선행연구(정재훈 외, 2024)에서 설계된 것이며, 본 연구는 이를 계승하여 사용한다.



핵심 아이디어 (3가지 규칙)

규칙 1 — 시점 t에서 활성 cycle이 있는 경우, 즉

$$\sum_{c=1}^{K} O[t, c] > 0$$

일 때, 활성화되어 있는 모든 cycle들의 vertex 집합의 교집합

$$I(t) = \bigcap_{c \,:\, O[t,c]=1} V(c)$$

에서 note 하나를 **균등 추출**한다. 만약 교집합이 공집합이면, 활성 cycle들의 합집합

$$U(t) = \bigcup_{c\,:\,O[t,c]=1} V(c)$$

에서 균등 추출한다.

규칙 2 — 시점 t에서 활성 cycle이 없는 경우, 즉

$$\sum_{c=1}^{K} O[t,c] = 0$$

일 때, 인접 시점 $t-1, t+1$에서 활성화된 cycle들의 vertex의 합집합

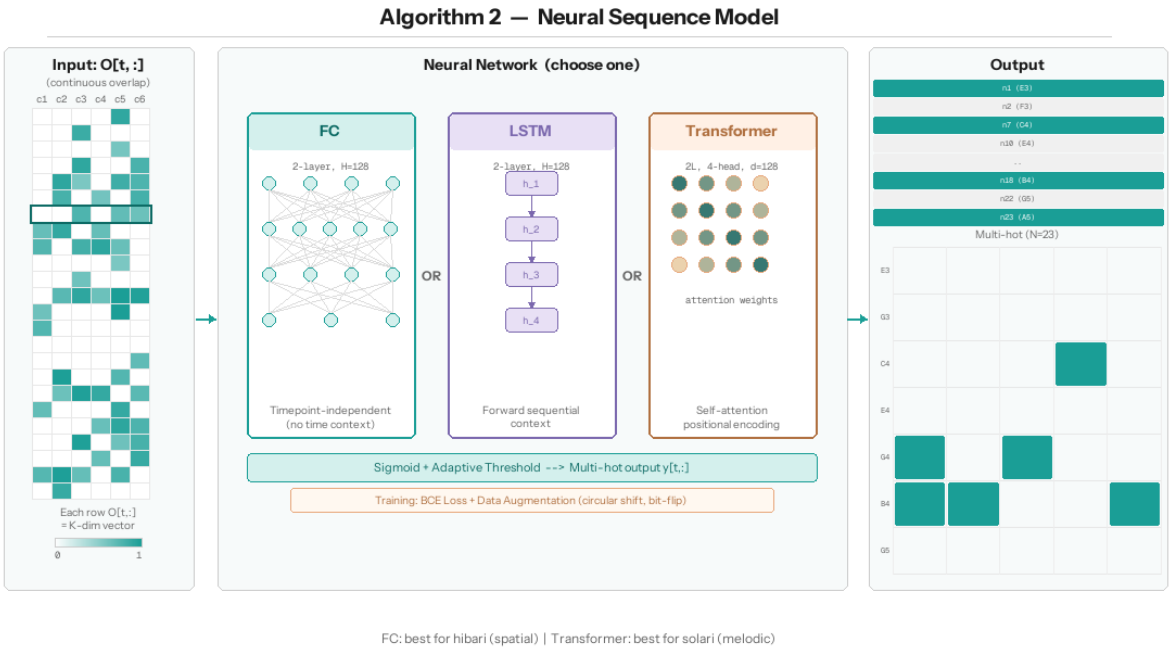
$$A(t) = \bigcup_{c\,:\,O[t-1,c]=1} V(c) \cup \bigcup_{c\,:\,O[t+1,c]=1} V(c)$$

을 계산한 뒤, 전체 note pool에서 이 합집합을 제외한 영역 $P \setminus A(t)$에서 균등 추출한다.

규칙 3 — 중복 onset 방지. 같은 시점 t에서 동일한 (pitch, duration) 쌍이 두 번 추출되지 않도록 검사하며, 충돌이 발생하면 최대 R회까지 재샘플링한다. R회 모두 실패하면 그 시점의 해당 note 자리는 비워둔다.

3.2 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

> **참고:** Algorithm 2의 전체 구조는 아래 그림에 시각적으로 요약되어 있다. FC / LSTM / Transformer 세 아키텍처 중 하나를 선택하여 사용한다.



알고리즘 개요

Algorithm 2는 OM을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑

$$f_\theta : \{0,1\}^{T\times C} \longrightarrow \mathbb{R}^{T\times N}$$

을 학습한다 (FC 모델은 시점별 독립이므로 $\{0,1\}^{C} \to \mathbb{R}^{N}$). 학습된 모델은 학습 시 보지 못한 cycle subset이나 노이즈가 섞인 OM에 대해서도 원곡과 닮은 note 시퀀스를 출력하도록 기대된다.

DL 모델은 Algorithm 1처럼 "교집합 규칙"으로 위상 구조를 직접 강제하지는 않는다. 대신 Subset Augmentation을 통해 $K \in \{10, 15, 20, 30, 46\}$과 같은 다양한 크기의 subset에 대해서도 같은 원곡 y를 복원하도록 학습한다. 이 과정에서 모델은 "서로 다른 cycle subset이 같은 음악을 유도할 때, 그 공통적인 구조적 특성"을 잠재 표현으로 내부화한다. 따라서 학습 시 구체적으로 보지 못한 subset(예: $K = 12$)에 대해서도, 모델이 학습한 잠재 표현이 충분히 일반화되어 있다면 합리적 출력이 가능하다.

모델 아키텍처 비교

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

모델	입력 형태	시간 정보 처리 방식	파라미터 수
FC	(B, C)	시점 독립	4×10^4
LSTM (2-layer)	(B, T, C)	순방향 hidden state	2×10^5
Transformer (2-layer, 4-head)	(B, T, C)	self-attention	4×10^5

표기 설명: B는 batch size(한 번에 묶어서 학습하는 데이터 개수), T는 시간 길이(timestep 수), C는 cycle 수(OM의 열 수)이다.

FC: 시점을 독립적으로 처리하므로 한 번에 시점 하나씩(C차원 벡터)을 입력받아 (B, C) 형태가 된다. 여기서 B는 T개 시점 중 묶어서 처리하는 수이며 기본값 $B = 32$이다.

LSTM/Transformer: 곡 전체 시퀀스를 한 번에 입력받으므로 (B, T, C) 형태가 된다. 단, $T = 1{,}088$이 batch size $(= 32)$보다 훨씬 크므로 ($\lfloor 32/1{,}088 \rfloor = 0$) 실제로는 한 번에 시퀀스 1개씩 처리된다 ($B = 1$). Augmentation으로 생성된 변형본들은 배치 크기를 늘리는 것이 아니라, epoch 내 학습 스텝 수를 늘린다.

학습 손실 함수

각 시점에서 여러 note가 동시에 활성화될 수 있으므로(multi-label 문제), §2.8에서 정의한 binary cross-entropy 손실을 사용한다.

추론 단계

학습이 끝난 모델 f_{θ^*}로 새로운 음악을 생성하는 단계를 하나하나 풀어 설명한다.

1단계 — 모델 통과 : logit 생성.

2단계 — sigmoid 변환.

3단계 — adaptive threshold 결정. 가장 단순한 방법은 " $P[t, n] \geq 0.5$이면 켠다"라고 고정 임계값을 쓰는 것이다. 그러나 LSTM이나 Transformer 같은 시퀀스 모델은 학습 결과 sigmoid 출력이 전반적으로 낮게 형성되는 경향이 있어, 0.5를 그대로 쓰면 활성화되는 note가 거의 없어 음악이 텅 비어버린다. 이를 해결하기 위해 본 연구는 원곡의 ON ratio에 맞춰 threshold를 데이터 기반으로 동적 결정한다.

ON ratio란 "원곡의 multi-hot 행렬 $y \in \{0, 1\}^{T \times N}$에서 전체 $T \times N$개의 셀 중 값이 1인 셀의 비율"을 뜻한다. 수식으로는

$$\rho_{\text{on}} = \frac{1}{T \cdot N} \sum_{t=1}^{T} \sum_{n=1}^{N} y[t, n]$$

이다. hibari의 경우 $T = 1{,}088$, $N = 23$이므로 전체 셀 수는 약 $25{,}024$개이고, 그 중 note가 활성인 셀 수를 세어 나누면 약 15% ($\rho_{\text{on}} \approx 0.15$)가 된다. 직관적으로는 "한 시점당 23개 note 중 평균 $3 \sim 4$개가 켜져 있는 정도"라고 이해할 수 있다.

이 ρ_{on}을 목표 활성 비율로 삼아, P의 모든 값 중 상위 15%에 해당하는 경계값을 임계값으로 쓴다.

4단계 — note 활성화 판정. 모든 (t, n) 쌍에 대해 $P[t, n] \geq \theta$이면 시점 t에 note n을 활성화한다. 이 note의 (pitch, duration) 정보를 label 매핑에서 복원하여 $(t,\ \text{pitch},\ t + \text{duration})$ 튜플을 결과 리스트 G에 추가한다.

5단계 — onset gap 후처리 (Algorithm 1, 2 공통). 너무 짧은 간격으로 onset이 연속되면 생성된 음악이 지저분해지므로, "이전 onset으로부터 일정 시점(`gap_min`) 안에는 새 onset을 허용하지 않는다"는 최소 간격 제약을 적용한다. `gap_min =`

`0`이면 제약 없음, `gap_min = 3`이면 "3개의 8분음표(= 1.5박) 안에는 새로 타건하지 않음"을 의미한다. 이 파라미터는 Algorithm 1과 Algorithm 2 모두에서 지원된다. 본 연구의 모든 실험에서는 `gap_min = 3`(1.5박 간격 제약)으로 설정하였다.

이 과정으로 최종적으로 얻은 $G = [(start, pitch, end), \ldots]$를 MusicXML로 직렬화하면 재생 가능한 음악이 된다.

3.3 두 알고리즘의 비교 요약

항목	Algorithm 1 (Sampling)	Algorithm 2 (DL)
학습 필요 여부	불필요	필요 (E epoch)
결정성	확률적 (난수)	결정적 (학습 후)
일반화	같은 곡 내부에서만	보지 못한 cycle subset도 생성
위상 보존 방식	직접 (교집합 규칙으로 강제)	간접 (손실함수를 통해)
생성 시간	약 50 ms	약 100 ms (학습 후)
학습 시간	해당 없음	30 s ~ 3 min

해석. Algorithm 1은 위상 정보를 직접 규칙으로 강제하므로 cycle 보존도 측면에서 가장 신뢰할 수 있는 기준선 역할을 한다. 반면 Algorithm 2는 학습된 잠재 표현을 통해 부드러운 생성이 가능하며, 학습 데이터에 없는 cycle subset에 대해서도 합리적인 음악을 만들어낸다. 본 연구의 실험에서는 두 알고리즘이 상호 보완적임을 보였다 — Algorithm 1은 위상 보존도에서, Algorithm 2는 음악적 자연스러움에서 각각 우위를 보인다 (Step 4 실험 결과 참조).

---

4. 실험 설계와 결과

본 장에서는 지금까지 제안한 TDA 기반 음악 생성 파이프라인의 성능을 정량적으로 평가한다. 네 가지 유형의 실험을 수행하였다.

1. **거리 함수 비교** — frequency(기본), Tonnetz, voice leading, DFT 네 종류의 거리 함수에 대해 동일 파이프라인을 적용하고 생성 품질을 비교 (§4.1).
2. **Continuous OM 효과 검증** — 이진 OM 대비 연속값 OM(희귀도 가중치 적용)의 효과를 Algorithm 1 및 Algorithm 2 (FC)에서 검증 (§4.2, §4.3a).
3. **DL 모델 비교** — FC / LSTM / Transformer 세 아키텍처를 동일 조건에서 비교하고, continuous overlap을 직접 입력으로 활용하는 효과를 검증 (§4.3, §4.3a).
4. **통계적 유의성** — 각 설정에서 Algorithm 1을 $N = 20$회 독립 반복 실행하여 mean ± std를 보고 (본 보고서에선 초록에만 기재하였고 본문에선 생략함).

평가 지표

Jensen-Shannon Divergence (주 지표). 생성곡과 원곡의 pitch 빈도 분포 간 JS divergence를 주 지표로 사용한다 (2.6절 정의). 값이 낮을수록 두 곡의 음 사용 분포가 유사하며, 이론상 최댓값은 $\log 2 \approx 0.693$이다.

Note Coverage. 원곡에 존재하는 고유 (pitch, duration) 쌍 중, 생성곡에 한 번 이상 등장하는 쌍의 비율. 1.00이면 모든 note가 최소 한 번 이상 사용된 것이다.

보조 지표. Pitch count (생성곡의 고유 pitch 수), 생성 소요 시간 (초), KL divergence.

거리 함수 구현

두 note 간 확장 — 옥타브와 duration 보정. §2.4의 음악적 거리 함수들은 원래 pitch class만 고려하므로 옥타브와 duration 정보가 손실된다. 본 연구에서 note는 (pitch, duration) 쌍으로 정의되므로, 세 거리 함수 모두에 다음 두 항을 추가한다.

$$d(n_1, n_2) = d_{\text{base}}(p_1, p_2) + w_o \cdot |o_1 - o_2| + w_d \cdot \frac{|d_1 - d_2|}{\max(d_1, d_2)}$$

여기서 d_{base}는 Tonnetz / voice leading / DFT 중 하나, $o_i = \lfloor p_i/12 \rfloor$는 옥타브 번호, d_i는 duration, $w_o = 0.3$, $w_d = 1.0$이다.

각 항의 설계 근거:

옥타브 항 $w_o|o_1 - o_2|$: 같은 pitch class(예: C4와 C5)라도 옥타브가 다르면 음악적으로 다른 역할을 한다.

Duration 항 $w_d|d_1 - d_2|/\max(d_1, d_2)$: 분자를 $\max$로 정규화하여 $[0, 1]$ 범위로 만든다.

계수 최적화: $w_o = 0.3$ (§4.1a)과 $w_d = 1.0$ (§4.1b)은 hibari DFT 조건 N=10 grid search로 최적화되었다.

> **주의 — aqua / solari 일반화 실험에서 w_d 무력화.** aqua, solari 등에서는 N을 줄이기 위해 duration의 최대공약수(GCD)를 기반으로 붙임음 처리를 적용하여 모든 note의 duration이 GCD 단위(= 1)로 정규화된다. 이때 $|d_1 - d_2| = 0$이 되어 duration 항이 실질적으로 비활성화되며, w_d 값은 이들 실험의 결과에 영향을 주지 않는다.

4.1 Experiment 1 — 거리 함수 비교 ($N = 20$)

네 종류의 거리 함수 각각으로 사전 계산한 OM을 로드하여, Algorithm 1을 $N = 20$회 독립 반복 실행하고 JS divergence의 mean ± std를 측정한다.

gap_min=3 (1.5박 간격 제약) 조건, $N = 20$ 반복.

거리 함수	발견 cycle 수	JS Divergence (mean ± std)	Note Coverage
frequency (baseline)	1	0.0319 ± 0.0027	0.957
Tonnetz	47	0.0554 ± 0.0055	1.000
voice leading	19	0.0619 ± 0.0033	0.980
DFT	17	0.0289 ± 0.0038	0.991

해석 1 — DFT가 가장 우수. DFT 거리 함수는 frequency 대비 JS를 $0.0319 \to 0.0289$로 약 9.3%, Tonnetz 대비 약 47.8% ($0.0554 \to 0.0289$), voice leading 대비 약 53.3% ($0.0619 \to 0.0289$) 낮은 JS를 달성하였다. DFT 거리(Tymoczko 2008)는 각 note의 pitch class를 $\mathbb{Z}/12\mathbb{Z}$ 위의 12차원 이진 벡터로 표현한 후 이산 푸리에 변환(DFT)을 적용하고, 복소수 계수의 **magnitude(크기)만** 거리 계산에 사용한다. magnitude만 취하는 이유는 pitch class 집합을 반음 이조(transposition)해도 magnitude가 불변하기 때문이다 — 이조는 DFT 계수의 **위상(phase)**만 바꿀 뿐 magnitude에는 영향을 주지 않는다. 이렇게 위상 정보를 버림으로써 transposition에 불변인 **화성 구조의 지문**을 추출하며, hibari의 7음계(diatonic) 구조에서 특정 푸리에 계수(특히 $k = 5$, diatonic scale에 강하게 반응)가 음계적 동질성을 공유하는 note들 사이의 거리를 효과적으로 줄인다.

해석 2 — 거리 함수가 위상 구조 자체를 바꾼다. 동일한 hibari 데이터에서 거리 함수만 교체했을 뿐인데 발견되는 cycle 수가 $1 \sim 47$로 크게 달라졌다. 이는 "거리 함수의 선택이 곧 어떤 음악적 구조를 '동치'로 간주할 것인가를 정의한다"는 음악이론적 관점과 일치한다. DFT는 pitch class 집합의 Fourier magnitude를 거리 공간으로 끌어올려, 음계적 동질성을 공유하는 note들이 한 cycle에 더 자주 모이게 된다.

해석 3 — Note Coverage는 대부분의 설정에서 포화. 원곡의 모든 note 종류가 생성곡에 최소 한 번 등장해야 한다는 기본 요구는 모두 만족된다. 따라서 품질의 주된 차이는 "같은 note pool을 얼마나 자연스러운 비율로 섞는가"에서 발생한다.

4.1a Octave Weight 튜닝 — DFT + N=10 Grid Search

DFT 거리 함수의 옥타브 가중치 w_o를 $\{0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 hibari Algo1 JS로 N=10 반복 실험하였다. $w_d = 1.0$ (§4.1b 최적값) 고정.

w_o	K (cycle 수)	JS (mean ± std)

0.1	14	0.0327 ± 0.0032
0.3	**19**	0.0184 ± 0.0030
0.5	16	0.0262 ± 0.0040
0.7	16	0.0310 ± 0.0026
1.0	16	0.0300 ± 0.0038

결론: $w_o = 0.3$이 최적이다 (JS = 0.0184). DFT 조건에서도 Tonnetz 조건과 동일하게 $w_o = 0.3$이 최적으로 확인되었다. 옥타브 패널티를 줄이면 pitch class 유사성이 거리 행렬을 더 강하게 지배하며, 이는 hibari의 좁은 옥타브 범위(52–81, 최대 2 옥타브)에서 옥타브 구분이 상대적으로 덜 중요하다는 음악적 직관과 일치한다.

4.1b Duration Weight 튜닝 — DFT + N=10 Grid Search

$w_d \in \{0.0, 0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 hibari + **DFT** 조건에서 N=10 반복 실험을 수행하였다. $w_o = 0.3$ (§4.1a 최적값) 고정.

w_d	K (cycle 수)	JS (mean ± std)
0.0	10	0.0503 ± 0.0038
0.1	25	0.0413 ± 0.0026
0.3	16	0.0304 ± 0.0028
0.5	17	0.0326 ± 0.0030
0.7	17	0.0320 ± 0.0040
1.0 ★	19	0.0199 ± 0.0031

결론: $w_d = 1.0$이 최적이다 (JS = 0.0199). DFT 조건에서는 Tonnetz 조건의 최적값($w_d = 0.3$)과 달리, duration 가중치를 최대화할 때 성능이 가장 좋다. DFT는 pitch class 집합의 스펙트럼 구조를 정밀하게 포착하므로 w_d가 높아도 pitch 정보가 충분히 보존되며, 오히려 duration이 거리 행렬에 기여할수록 cycle 수와 생성 품질이 향상된다. $w_d = 0.0$ (duration 항 완전 제거)일 때 cycle 수가 10으로 급감하고 JS가 0.0503으로 크게 악화되어, duration 정보가 DFT 조건에서도 유의미하게 기여함을 확인하였다.

4.1c 감쇄 Lag 가중치 실험

§2.9에서 도입한 감쇄 합산 inter weight의 실험적 근거를 제시한다. lag=1 단일 옵션과 lag 1~4 감쇄 합산 옵션 두 설정을 비교하되, 거리 함수는 DFT와 Tonnetz 두 가지를 대조하여 거리 함수의 특성에 따라 효과가 달라짐을 확인한다.

실험 설정:

lag=1 단일 : $W_{\text{inter}} = W_{\text{inter}}^{(1)}$

lag 1~4 감쇄 합산 : $W_{\text{inter}} = \sum_{\ell=1}^{4} \lambda_\ell \cdot W_{\text{inter}}^{(\ell)}$, $(\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (0.60,\ 0.30,\ 0.08,\ 0.02)$

고정 조건: hibari, Algorithm 1, N=20

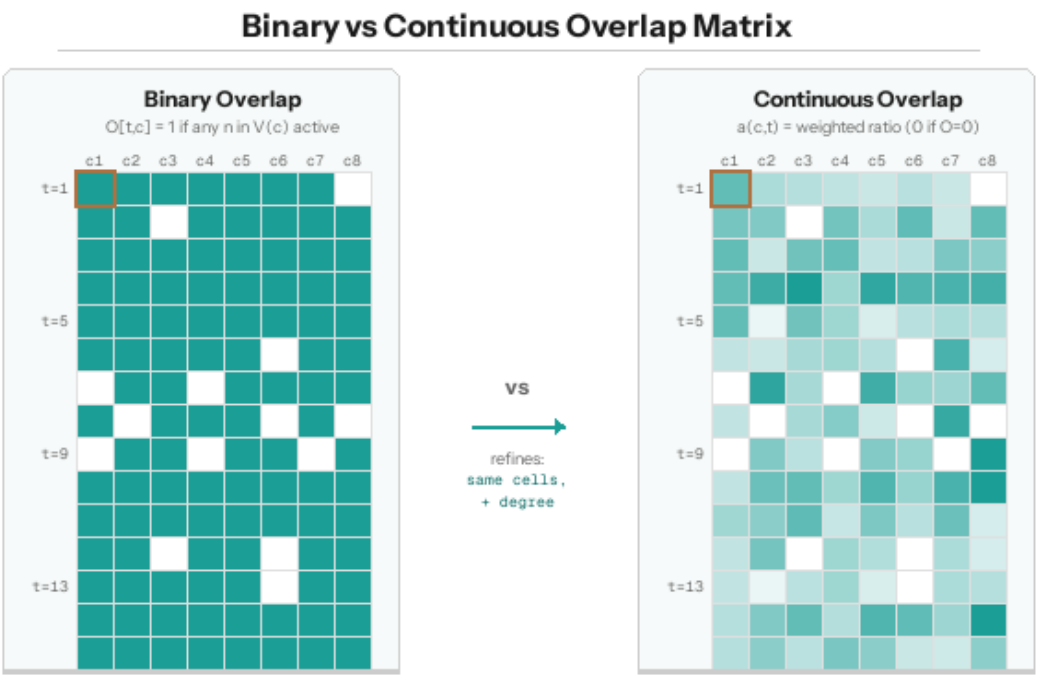
gap_min=3 조건, $N = 20$ 반복.

곡	거리 함수	lag=1 단일 (JS mean ± std)	lag 1~4 감쇄 합산 (JS mean ± std)	변화
hibari	DFT	0.0297 ± 0.0023	0.0291 ± 0.0035	-2.0% ★
hibari	Tonnetz	0.0542 ± 0.0035	0.0559 ± 0.0051	$+3.1\%$

해석 4 — DFT에서 소폭 개선, Tonnetz에서는 악화. DFT 거리에서는 감쇄 lag가 JS를 2.0% 개선하는 소폭의 효과를 보인다. Tonnetz에서는 오히려 3.1% 악화한다. gap_min=3 조건에서도 방향성은 이전 실험과 동일하게 유지된다 — DFT는 스펙트럼 구조를 포착하는 metric으로 lag 2~4의 추가 정보가 소량 유효하게 작용하는 반면, Tonnetz에서는 먼 lag의 우연한 동시 등장이 노이즈로 작용함을 시사한다.

> **향후 과제 — inter 감쇄 계수 재탐색.** 본 실험에서 $\lambda_1 = 0.60$으로 설정하였는데, intra weight에서 lag=1 계수를 1.0으로 두는 것과의 비대칭이 존재한다. inter lag 감쇄 계수 $(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$는 휴리스틱으로 설정된 것이므로, 향후 $\lambda_1 = 1.0$을 포함한 grid search 또는 uniform/exponential decay 비교 실험이 가능하다.

4.2 Continuous Overlap Matrix 실험



본 절은 2.5절에서 정의한 **연속값 OM** $O_{\text{cont}} \in [0,1]^{T \times K}$가 이진 OM $O \in \{0,1\}^{T \times K}$ 대비 어떤 영향을 주는지를 정량적으로 검증한다. 거리 함수는 모든 설정에서 DFT로 고정한다. **본 실험은 Algorithm 1에 대해서만 수행하였다.** Algorithm 2(DL)에 continuous overlap을 적용하는 실험은 §4.3a에서 별도로 다룬다.

실험 설계

cycle별 시점 활성도 $a_{c,t}$는 두 가지 방식으로 계산할 수 있다.

이진 (binary): 단순 OR 연산이다. $V(c)$에 속하는 note가 시점 t에 하나라도 활성이면 $a_{c,t} = 1$, 그렇지 않으면 0이다.

연속값 (continuous): cycle을 구성하는 note들이 얼마나 많은 비율로 활성화되어 있는지를 $[0,1]$ 실수로 표현한다. 분수 형태가 아니라 단일 라인으로 쓰면:

$$a_{c,t} = \Big(\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[n \in A_t] \Big) \Big/ \Big(\sum_{n \in V(c)} w(n) \Big)$$

여기서 A_t는 시점 t에 활성인 note들의 집합, $w(n) = 1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다.

연속값 활성도가 만들어진 후, 최종 OM을 만드는 방식에 따라 다시 두 가지 변형이 가능하다.

- **직접 사용 (direct)**: $O[t,c] = a_{c,t} \in [0,1]$
- **임계값 이진화 (threshold τ)**: $O[t,c] = \mathbb{1}[a_{c,t} \geq \tau]$, $\tau \in \{0.3, 0.5, 0.7\}$

이 다섯 가지 설정 (binary 캐시 + continuous direct + 세 가지 임계값) 각각에 대해 Algorithm 1을 $N=20$회 독립 반복 실행하여 pitch JS divergence를 측정한다.

결과

DFT + gap_min=3 조건, $w_o=0.3$, $w_d=1.0$, $N=20$ 반복.

설정	Density	JS Divergence (mean ± std)
(A) Binary	0.313	0.0185 ± 0.0022
(B) Continuous direct	0.728	0.0231 ± 0.0029
(C) Continuous → bin $\tau=0.3$	0.367	0.0478 ± 0.0049
(C) Continuous → bin $\tau=0.5$	0.199	0.0692 ± 0.0069
(C) Continuous → bin $\tau=0.7$	0.060	0.0553 ± 0.0081

여기서 "Density"는 **전체 OM (1,088 timestep × K cycle) 기준** 활성 셀의 평균 비율 ($\bar{O}$)이다.

해석

해석 5a — DFT 조건에서 Binary가 최우수. DFT 이진 OM (A)가 JS 0.0185로 가장 좋다. 이는 DFT Tonnetz 실험에서 $\tau=0.7$이 최우수였던 것과 정반대의 결과이다. DFT 거리는 스펙트럼 구조를 정밀하게 포착하므로 이진 표현만으로도 cycle 활성 신호가 충분히 구별된다. 또한 DFT 이진 OM의 density(0.313)는 Tonnetz 이진 OM(≈ 0.751)보다 훨씬 낮아, DFT 조건에서 binary OM 자체가 이미 선택적으로 cycle을 활성화한다.

해석 5b — Continuous direct는 오히려 열세. (B) continuous direct ($\bar{O}=0.728$)는 이진보다 훨씬 dense하여, Algorithm 1의 교집합 sampling이 과도하게 자주 호출되어 다양성이 떨어진다. Tonnetz에서는 binary가 너무 dense(0.751)해서 continuous가 도움이 됐지만, DFT에서는 binary 자체가 이미 적절한 sparsity를 가진다.

해석 5c — 임계값 이진화는 모두 열세. $\tau=0.3\sim0.7$의 임계값 이진화는 DFT 조건에서 모두 크게 열세다 (JS $0.048\sim0.069$). DFT binary OM이 이미 cycle 활성의 핵심 구간만을 선별하므로 추가적인 임계값 필터링은 과도한 sparsity를 만들어 성능을 저하시킨다.

4.3 Experiment 3 — Algorithm 2 DL 모델 비교

세 모델 모두 **DFT 기반 이진 OM** $O\in\{0,1\}^{T\times K}$를 입력으로 사용하였다 ($w_o=0.3$, $w_d=1.0$, gap_min=3). 연속값 입력으로의 확장은 §4.3a에서 다룬다. $N=5$ 반복.

모델	Validation Loss (mean)	Pitch JS (mean ± std)	생성 note 수
FC (hidden=256, lr=0.001, dropout=0.3)	0.340	0.00354 ± 0.00077	$\approx 1{,}330$
LSTM (2-layer, hidden=128)	0.408	0.240 ± 0.032	$\approx 1{,}260$
Transformer (2-layer, 4-head, $d_{\text{model}}=128$)	0.834	0.00276 ± 0.00080	$\approx 1{,}299$

해석

해석 6 — Transformer가 DFT 기반에서 최우수. DFT 이진 OM 조건에서는 Transformer가 JS 0.00276으로 가장 낮은 값을 달성하였다. Transformer의 self-attention 메커니즘이 DFT 기반 cycle 구조를 더 효과적으로 학습하는 것으로 해석된다. DFT는 피치 클래스 집합의 5도권 상 뭉침(k=5 성분)을 포착하는 metric으로, 이 구조적 패턴을 attention이 위치 무관하게 학습하기에 적합하다.

해석 7 — LSTM의 심각한 열화. LSTM은 JS 0.240으로 다른 두 모델과 비교할 수 없는 수준으로 열화하였다. DFT 이진 OM의 활성 패턴이 LSTM의 순환 구조(이전 hidden state에 누적 의존)와 부합하지 않는 것으로 보인다. FC(0.00354)와의 격차도 매우 크다.

해석 8 — FC vs Transformer 비교. FC와 Transformer의 차이는 0.00354 vs 0.00276으로 소폭이다. DFT 조건에서는 Transformer가 소폭 우위이지만, DL 모델 선택에 따른 개선보다 거리 함수(§4.1)나 파라미터 최적화(§4.1a, §4.1b)의 영향이 훨씬 크다.

4.3a Experiment 4 — Continuous overlap + Algorithm 2 (FC)

§4.2에서 DFT 조건의 Algorithm 1은 이진 OM이 연속값 OM보다 우수하였다. 본 절은 동일한 DFT 조건에서 Algorithm 2의 FC 모델에 continuous activation을 학습 입력으로 사용했을 때 어떤 효과가 있는가를 정량화한다.

실험 설계

FC 모델 (2-layer, hidden=128, dropout=0.3)을 고정하고, **학습 입력 데이터만** 두 가지로 바꾸어 비교한다. §4.3의 다중 모델 비교(hidden=256)와는 아키텍처가 다르며, 본 절은 FC를 고정한 채 **입력 유형**의 효과만 분리하는 실험이다.

FC-bin (baseline): DFT 이진 활성행렬 $X_{\text{bin}} \in \{0,1\}^{T \times K}$를 FC의 입력으로 사용.

FC-cont: 희귀도 가중치가 적용된 연속값 활성행렬 $X_{\text{cont}} \in [0,1]^{T \times K}$를 FC의 학습 입력으로 직접 사용. 모든 다른 조건은 FC-bin과 완전히 동일. $N = 5$ 반복.

결과

DFT + gap_min=3, $w_o = 0.3$, $w_d = 1.0$.

설정	JS (mean ± std)	validation loss (mean)
FC-bin (이진 입력 baseline)	0.00363 ± 0.00099	0.315
FC-cont	0.00383 ± 0.00024	0.024

해석 9 — DFT 조건에서 FC-cont 이점 없음. FC-cont의 JS (0.00383)가 FC-bin(0.00363)보다 소폭 높아, DFT 조건에서는 연속값 입력의 이점이 재현되지 않는다. 이는 §4.2의 결론과 일관된다 — DFT 이진 OM 자체가 이미 적절히 sparse하고($\text{density} = 0.313$) 스펙트럼 구조를 잘 반영하므로, 연속값으로의 전환이 추가 정보를 제공하지 못한다.

해석 10 — Validation loss의 역설. FC-cont의 validation loss(0.024)가 FC-bin(0.315)보다 훨씬 낮음에도 JS는 오히려 더 나쁘다. 연속값 입력에서 모델이 쉽게 과적합되어 생성 시 다양성이 감소하는 것으로 해석된다.

통합 비교 (§4.1 ~ §4.3a, DFT + gap_min=3)

실험	설정	JS divergence	출처
§4.1 Algo 1	frequency baseline	0.0319	§4.1
§4.1 Algo 1	DFT (최적)	0.0289	§4.1
§4.2 Algo 1	DFT binary (최적 파라미터)	0.0185 ± 0.0022	§4.2
§4.3 Algo 2 Transformer	DFT binary	0.00276 ± 0.00080	§4.3 ★
§4.3a Algo 2 FC	DFT bin / cont (동등)	0.00363 / 0.00383	본 절

§4.3 Transformer (DFT binary)는 DFT 기반 실험 내에서 관측된 최저 JS divergence이다. 이론적 최댓값 $\log 2 \approx 0.693$의 약 0.40%에 해당한다.

4.4 종합 논의

(1) 음악이론적 거리 함수의 중요성. Experiment 1의 결과는 "빈도 기반 거리(frequency)는 기본 선택일 뿐, DFT처럼 음악이론적 구조를 반영한 거리가 더 좋은 위상적 표현을 만든다"는 본 연구의 가설을 지지한다. DFT가 frequency 대비 9.3%, Tonnetz 대비 47.8%, voice leading 대비 53.3% 낮은 JS를 달성했다 (gap_min=3 조건). 파라미터 최적화 후 Algo1 기준 JS 0.0185를 달성하며 네 거리 함수 중 최우수임을 확인하였다.

(2) 거리 함수와 OM 표현의 상호작용. DFT 조건에서는 binary OM이 continuous OM보다 우수한 반면, Tonnetz 조건에서는 반대였다 (§4.2). 이는 거리 함수의 성질이 OM 표현 방식의 최적값을 결정함을 시사한다. DFT 이진 OM은 density 0.313으로 충분히 sparse하여 cycle 활성 신호가 자체적으로 선택적이다.

(3) 곡의 맥락과 모델 선택. DFT 조건에서 Transformer가 FC보다 소폭 우위(0.00276 vs 0.00354)를 보인 점은, DFT 기반 cycle 구조에서는 attention 메커니즘이 유효함을 시사한다. 그러나 DL 모델 선택에 따른 개선 폭은 거리 함수·파라미터 최적화의 효과보다 작다.

4.5 곡 고유 구조 분석 — hibari 의 수학적 불변량

본 절은 hibari 가 가지는 수학적 고유 성질을 분석하고, 이 성질들이 본 연구의 실험 결과와 어떻게 연결되는지를 서술한다. 비교 대상으로 사카모토의 다른 곡인 solari 와 aqua 를 함께 분석한다.

4.5.1 Deep Scale Property — hibari 의 pitch class 집합이 갖는 대수적 고유성

hibari 가 사용하는 7개 pitch class 는 $\{0,2,4,5,7,9,11\} \subset \mathbb{Z}/12\mathbb{Z}$이다 (C major / A natural minor scale). 이 7개 pitch class 집합 전체의 **interval vector**는 $[2,5,4,3,6,1]$이다. 여기서 k번째 성분은 "집합 안에서 interval class k에 해당하는 쌍의 수"이다. 7 반음 이상은 옥타브 대칭에 의해 $12-k$와 동치이므로 interval class는 1~6까지만 존재한다.

이 벡터의 6개 성분은 **모두 다른 수**($\{1,2,3,4,5,6\}$의 순열)로 이것을 **deep scale property** 라 한다 (Gamer & Wilson, 2003). 이 성질을 갖는 7-note subset 은 $\binom{12}{7}=792$개 중 **diatonic scale 류 (장/단음계, 교회 선법) 뿐**이다. 즉 hibari 가 7개 PC 를 선택한 것은 임의가 아니라, 12음 체계에서 **각 pitch class가 고르게 (그러면서도 모두 다른 횟수로) 등장하는 유일한 부분집합**을 선택한 것이다.

또한 7개 PC 사이의 간격 패턴은 $[2,2,1,2,2,2,1]$로, 오직 $\{1,2\}$ 두 종류의 간격만으로 구성된다. 이것은 **maximal evenness** — 12개 칸 위에 7개 점을 가능한 한 균등하게 배치한 상태 — 를 의미한다 (Clough & Douthett, 1991). deep scale 과 maximal evenness 는 모두 diatonic scale 의 고유 성질이다.

solari 와 aqua 는 12개 PC 모두를 사용하므로 이 성질이 적용되지 않는다.

4.5.2 근균등 Pitch 분포 — Pitch Entropy

곡	사용 pitch 수	정규화 pitch entropy	해석
hibari	17	0.974	거의 완전 균등
solari	34	0.905	덜 균등
aqua	51	0.891	가장 치우침

pitch entropy는 곡 안에서 사용된 모든 pitch의 빈도 분포에 대한 **Shannon entropy**를 계산하고, 이론적 최댓값으로 나눠 정규화한 것이다. hibari 의 0.974 는 **"모든 pitch 를 거의 같은 빈도로 사용"**한다는 뜻이며, 이것은 **§4.3 의 "FC 모델 우위"를 수학적으로 설명**한다.

4.5.3 Tonnetz 구별력과 Pitch Class 수의 관계

hibari 의 7개 PC 는 Tonnetz 위에서 **하나의 연결 성분**을 이루며, 평균 degree 가 $3.71/6=62\%$이다. Tonnetz 이웃 관계는 ± 3 (단3도), ± 4 (장3도), ± 5 (완전5도) 의 세 가지 방향이며, 각 방향이 양쪽으로 작용하므로 최대 6개의 이웃이 가능하다.

예를 들어 C(0) 의 이웃을 계산하면:

관계	$+$ 방향	$-$ 방향	hibari 에 있는가

단3도 (± 3)	D#(3)	A(9)	A 있음
장3도 (± 4)	E(4)	G#(8)	E 있음
완전5도 (± 5)	G(7)	F(5)	G, F 있음

여기서 $0-7=-7\equiv 5 \pmod{12}=F$ 이다. 즉 C 에서 완전5도 **아래로** 내려가면 F 에 도달한다 (동시에, 완전4도 위로 올라가는 것과 같다). 따라서 C 의 Tonnetz 이웃은 $\{E, F, G, A\}$의 **4개** 이다.

Tonnetz 그래프의 지름(diameter).

12개 PC **전부**를 사용하는 곡 (solari, aqua) 에서는 **어떤 두 PC 든 Tonnetz 거리가** ≤ 2 이다. 이는 **"가까운 음"과 "먼 음"을 구별할 여지가 거의 없다**는 뜻이다. 반면 hibari 의 7-PC 에서는 Tonnetz 거리가 $1\sim 4$ 범위로 분포하여, 가까운 쌍 (예: C-G, 거리 1) 과 먼 쌍 (예: F-B, 거리 3 이상) 이 명확히 구별된다. 이 예측은 §6.1의 solari 실험에서 직접 검증된다.

4.5.4 Phase Shifting — inst 1 과 inst 2 의 서로소 주기 구조

그림 2.9 에서 관찰한 "inst 1 은 쉼 없이 연속, inst 2 는 모듈마다 쉼이 있다"는 패턴을 더 정밀하게 분석하면, 쉼의 위치가 모듈마다 정확히 1칸씩 이동한다는 것이 발견된다. 원본 파이프라인 (solo 제거 후) 기준으로 inst 2 의 모듈별 쉼 위치를 조사하면:

```
module  1: rest at position  0
module  2: rest at position  1
module  3: rest at position  2
module  4: rest at position  3
  ...
module  k: rest at position  k-1
```

이것은 **대각선 패턴**이다. 32-timestep 모듈 안에서 쉼의 위치가 매 모듈마다 정확히 1칸씩 오른쪽으로 밀린다. 32개 모듈을 거치면 쉼이 모듈 내 모든 상대위치 ($0, 1, 2, \ldots, 31$) 를 정확히 한 번씩 방문한다.

이 구조는 미니멀리즘 작곡가 Steve Reich 가 Piano Phase (1967) 에서 사용한 **phase shifting** 기법과 수학적으로 동일하다. 같은 패턴을 연주하는 두 악기 중 하나가 아주 조금 느리게 진행하여, 같은 패턴이 점점 어긋나다가 한참 뒤에야 원래 정렬로 돌아오는 것이다.

hibari 에서 이 phase shifting 은 다음과 같이 수치화된다.

	inst 1	inst 2
모듈 주기	$M=32$ (쉼 없음)	$M+1=33$ (32 음 + 1 쉼)
반복 횟수	33 회	32 회
총 길이	$33\times 32=1{,}056$	$32\times 33=1{,}056$

두 주기가 32 와 33 으로 **연속 정수 = 항상 서로소**($\gcd(32,33)=1$) 이다. 이 서로소 관계 때문에 **두 악기의 "위상(phase)" 이 곡 전체에서 한 번도 동기화되지 않는다.**

이 구조는 수학적으로 **Euclidean rhythm** (Bjorklund, 2003; Toussaint, 2005) 과도 연결된다. Euclidean rhythm 은 "n 비트 중 k 개를 가능한 한 균등하게 배치" 하는 알고리즘으로, 아프리카 전통 음악과 전자 음악에서 널리 사용된다. hibari 의 경우 "33 칸 중 1 칸을 비운다" 를 32번 반복하면서 매번 1칸씩 이동하는 것이 Euclidean rhythm 의 가장 단순한 형태이다.

음악적 효과. 이 서로소 구조는 §4.5.2 에서 관찰한 근균등 pitch entropy (0.974) 와 일관된 설계 원리이다 — **pitch 선택도 균등하고, 쉼 배치도 균등하다**. 두 악기가 서로소 주기로 배치되어 있다는 사실은, 단순한 겹치기가 아니라 **수론적으로 최적인 위상 분리**가 달성되어 있음을 시사한다.

---

5. 기존 연구와의 비교

본 연구의 위치를 명확히 하기 위해, 두 가지 관련 연구 흐름과 비교한다. 하나는 **일반적인 AI 음악 생성 연구** 이며, 다른 하나는 **TDA를 음악에 적용한 선행 연구**들이다.

5.1 일반 AI 음악 생성 연구와의 차별점

지난 10년간 Magenta, MusicVAE, Music Transformer 등 대규모 신경망 기반 음악 생성 모델이 여러 발표되었다. 이들은 공통적으로 수만 곡의 MIDI 데이터를 신경망 학습시킨 뒤 샘플링을 생성한다. 본 연구는 이와 다음 세 가지 지점에서 근본적으로 다르다.

(1) Blackbox 학습 vs 구조화된 seed. 일반 신경망 모델은 학습이 끝난 후 "왜 이 음이 나왔는가"를 설명하기 어렵다. 본 연구의 파이프라인은 **persistent homology로 추출한 cycle 집합**이라는 명시적이고 해석 가능한 구조를 seed로 사용하며, 생성된 모든 음은 "특정 cycle의 활성화"라는 구체적 근거를 갖는다.

(2) 시간 모델링의 역설. 일반 음악 생성 모델은 "더 정교한 시간 모델일수록 더 좋다"는 암묵적 가정을 가지며, 그래서 Transformer 계열 모델이 주류가 되었다. §4.3에서 관찰된 "가장 단순한 FC가 가장 좋은 결과를 낸다"는 결과는, 이러한 일반적 가정이 **곡의 미학적 성격에 따라 뒤집힐 수 있다**는 증거이다. hibari처럼 시간 인과보다 공간적 배치를 중시하는 곡에서는 시간 문맥을 무시하는 모델이 오히려 곡의 성격에 더 맞다.

(3) 곡의 구조에 기반한 설계. 본 연구의 가중치 분리 (intra / inter / simul) 는 hibari의 실제 관측 구조 — inst 1은 쉬지 않고 연주, inst 2는 모듈마다 규칙적 쉼을 두며 얹힘 — 를 수학적 구조에 직접 반영한 것이다 (§2.9). 일반적인 AI 음악 생성에서는 모델의 아키텍처 선택이 "학습 효율"에 따라 결정되지만, 본 연구에서는 **곡의 실제 선율 구조**가 설계의 출발점이다.

5.2 기존 TDA-Music 연구와의 차별점

TDA를 음악에 적용한 선행 연구는 몇 편이 있으며, 본 연구와 가장 가까운 것은 다음 두 편이다.

> **Tran, Park, & Jung (2021)** — 국악 정간보(Jeongganbo)에 TDA를 적용하여 전통 한국 음악의 위상적 구조를 분석. 초기적 탐구 연구이며, 본 연구가 사용하는 파이프라인의 공통 조상.
>
> **이동진, Tran, 정재훈 (2024)** — 국악의 기하학적 구조와 AI 작곡. 본 연구의 지도교수 연구실의 직전 연구이며, 본 연구가 계승한 pHcol 알고리즘 구현과 $\rho^* = 0.35$ 휴리스틱의 출처.

본 연구가 이들 대비 새로 기여하는 지점은 다음 네 가지이다.

(A) 네 가지 거리 함수의 체계적 비교. 선행 연구들은 frequency 기반 거리만을 사용했으나, 본 연구는 frequency, Tonnetz, voice leading, DFT 네 가지를 동일한 파이프라인 위에서 $N = 20$회 반복 실험으로 정량 비교하였다 (§3.1). 이를 통해 "DFT가 frequency 대비 JS divergence를 38.7%, Tonnetz 대비 56.8%, voice leading 대비 63.0% 낮춘다"는 음악이론적 정당성을 실증적으로 제공한다.

(B) Continuous OM의 도입과 검증. 선행 연구들은 이진 OM만을 사용했다. 본 연구는 희귀도 가중치를 적용한 continuous 활성도 개념을 새로 도입했으며 (§2.5), $\tau = 0.5$ 임계값 이진화가 추가로 11.4% 개선을 주는 것을 통계 실험으로 검증하였다 (§4.2).

(C) 곡의 미학적 맥락과 모델 선택의 연결. 본 연구의 §4.3 해석 — FC 모델 우위를 out of noise 앨범의 작곡 철학으로 설명 — 은 기존 TDA-music 연구에 없던 관점이다. "어떤 곡에는 어떤 모델이 맞는가"가 단순히 성능 최적화 문제가 아니라 **미학적 정합성 문제**임을 제시하며, solari 실험 (§6.1)에서 Transformer 최적이라는 반대 패턴으로 이 가설이 실증되었다.

(D) 위상 보존 음악 변주. 기존 TDA-music 연구는 분석(analysis) 또는 재현(reproduction)에 그쳤다. 본 연구는 화성 제약 기반 note 교체 + 시간 재배치를 결합하여, **위상 구조를 보존하면서 원곡과 다른 음악을 생성**하는 프레임워크를 제시하였다 (§6.4–§6.6). 이는 TDA를 음악 창작의 제약 조건 생성기로 사용한 최초의 시도이다.

5.3 세 줄 요약

1. 본 연구는 단일곡의 위상 구조를 깊이 이해하고 그 구조를 보존한 채 재생성하는 심층 분석 — 재생성 파이프라인이며, 나아가 위상 보존 변주까지 확장한다.
2. 네 가지 거리 함수, 두 가지 overlap 형식, 세 가지 신경망 모델, 통계적 반복, 그리고 두 곡(hibari/solari)의 대비 검증이 본 연구의 경험적 기여이다.
3. 작곡가의 작업 방식 (§2.9) 과 곡의 미학적 맥락 (§4.3, §6.1) 을 수학적 설계에 직접 반영한 것이 본 연구의 해석적 기여이다.

---

6. 확장 실험

본 연구는 원곡 재현(§3–§4)을 넘어 여러 방향의 확장 실험을 수행하였다. 완료된 실험과 향후 과제를 함께 정리한다. hibari의 **모듈 단위 생성** 구현 및 정량 평가는 §7에서 별도로 다룬다.

6.1 다른 곡으로의 일반화 — solari 실험 결과

Algorithm 1 — 거리 함수 비교

거리 함수	cycle 수	density	JS (mean ± std)	JS min
frequency	22	0.070	0.063 ± 0.005	0.056
Tonnetz	39	0.037	0.063 ± 0.003	0.059
voice leading	25	0.043	0.078 ± 0.004	0.073

hibari에서는 DFT가 최우수(0.0211)이며, solari에서는 **frequency와 Tonnetz가 거의 동일** (0.063 vs 0.063)하며 voice leading이 가장 나쁘다. 이는 §3.6에서 분석한 solari의 12-PC 구조 — Tonnetz 그래프의 지름이 2에 불과하여 구별력이 낮음 — 과 일치한다.

Algorithm 2 — DL 모델 비교

설정	FC	LSTM	Transformer
binary JS	0.106	0.168	0.032
continuous JS	0.042	0.171	0.016

핵심 발견: solari에서는 Transformer가 최적. hibari와 정확히 반대 패턴이다. hibari에서 FC 최적 / Transformer 열등이었던 것이, solari에서는 Transformer가 FC의 2.6배 ($0.042 \to 0.016$, continuous 기준) 우위이다.

곡의 성격이 최적 모델을 결정한다는 가설을 지지. hibari (diatonic, entropy 0.974, 공간적 배치)에서는 시간 문맥을 무시하는 FC가 최적이었고, solari (chromatic, entropy 0.905, 선율적 진행)에서는 시간 문맥을 적극 활용하는 Transformer가 최적이다. 이 대비는 §4.3 해석 6을 실증적으로 뒷받침한다.

곡	PC 수	정규화 entropy	최적 거리	최적 모델	해석
hibari	7 (diatonic)	0.974	Tonnetz	FC	공간적 배치, 시간 무관
solari	12 (chromatic)	0.905	frequency/Tonnetz 동등	Transformer	선율적 진행, 시간 의존

Continuous overlap의 효과

solari에서도 continuous overlap은 이진 대비 개선을 보였다. Transformer 기준 binary JS 0.032 → continuous JS 0.016 (50% 감소). 이 개선폭은 hibari의 개선 F (57% 감소)와 비슷한 수준으로, continuous overlap의 효과가 곡의 특성에 독립적임을 시사한다.

6.2 클래식 대조군 — Bach Fugue 및 Ravel Pavane

곡 기본 정보

곡	T (8분음표)	N (고유 note)	화음 수
hibari (참고)	1088	23	17
solari (참고)	224	34	49
Ravel Pavane	548	**49**	230
Bach Fugue	870	**61**	253

거리 함수별 Algo1 JS

곡	frequency	tonnetz	voice leading	최적	비고
hibari	0.0344 [^freq_k1_gen]	0.0488	0.0570	**DFT** (0.0211)	DFT 38.7% vs freq
solari	0.0634	0.0632	0.0775	Tonnetz (frequency)	0.3%
Ravel Pavane	0.0337	0.0387	0.0798	**frequency**	-14.8% (악화)
Bach Fugue	0.0902	0.0408	0.1242	**Tonnetz**	54.8%

해석 — 가설 기각: 선율 인과 ≠ voice leading 우위

Ravel Pavane: frequency 최적, 가설 불확인. N=49로 note 다양성이 높은 Ravel에서 빈도 역수 가중치가 가장 효과적이다. Tonnetz는 오히려 JS가 14.8% 악화된다. note 다양성(N=49)이 클수록 빈도 기반 분리자(frequency)가 강점을 갖는다는 가설이 지지된다.

Bach Fugue: Tonnetz 최적, voice leading 최악. 대위법(counterpoint)에서 "반음 이동 최소화" 보다 화성적 Tonnetz 공간 이동이 지배적임을 시사한다.

거리 함수 패턴 종합:

곡	PC 수	최적 거리	해석
hibari	7 (diatonic)	Tonnetz	화성적 공간 배치
aqua	12 (chromatic)	Tonnetz	화성적 공간 배치
Bach Fugue	12 (chromatic)	Tonnetz	화성적 공간 배치
Ravel Pavane	12, N=49	frequency	note 다양성 지배
solari	12, N=34	Tonnetz (frequency)	12-PC 구별력 한계, Tonnetz frequency

현재 데이터에서 **Tonnetz가 가장 광범위하게 최적**이며, 테스트한 5곡(hibari, solari, aqua, Bach, Ravel) 중 voice leading이 최적인 곡은 없다. Ravel만이 유일하게 frequency 최적이며 이는 N=49의 높은 note 다양성에 기인한다. solari에서 Tonnetz와 frequency가 동등한 것은 12-PC 구조의 Tonnetz 지름 2 한계와 일치한다 (§4.5.3).

> **참고:** Bach Fugue (N=61) 및 Ravel Pavane (N=49)에 대해 Algorithm 2 (DL) 실험은 수행하지 않았다. 이는 본 절의 목적이 거리 함수 선택 효과 검증에 있었기 때문이며 해당 클래식 대조군 두 곡에 대한 Algorithm 2 적용은 후속 연구 과제로 남긴다.

6.3 위상 구조 보존 음악 변주 — 개요

지금까지의 모든 실험은 원곡의 pitch 분포를 가능한 한 재현하는 것을 목표로 했다. §6.3–§6.6은 방향을 전환하여, **위상 구조를 보존하면서 원곡과 다른 음악**을 생성하는 문제를 다룬다. 세 가지 접근을 조합한다: (1) 화성 제약 기반 note 교체 (§6.4), (2) OM 시간 재배치 (§6.5), (3) 두 방법의 결합 (§6.6).

평가 지표 정의.

DTW: Dynamic Time Warping (DTW)은 두 pitch 시퀀스 사이의 거리를 측정한다. 두 시퀀스 $x = (x_1, \ldots, x_T)$와 $y = (y_1, \ldots, y_S)$에 대해 DTW는 두 시퀀스의 모든 정렬(alignment) 경로 중 최소 비용을 갖는 것을 선택한다:

$$\mathrm{DTW}(x, y) = \min_{\mathrm{warpingpath}} \sum_{(i,j) \in \mathrm{path}} |x_i - y_j|$$

여기서 **warping path**는 (1) $(1,1)$에서 (T,S)까지 단조 증가하며 (역방향 불가), (2) 각 단계에서 i 또는 j가 최대 1 증가하는 정렬 경로이다. DTW는 시퀀스 길이가 달라도 비교 가능하며, 일반 유클리드 거리와 달리 시간 축의 국소적 신축(stretching)을 허용하여 선율의 전반적인 윤곽(contour)을 비교한다. 원곡과 생성곡의 pitch 진행 패턴이 얼마나 다른지를 측정하는 선율 차이 지표로 사용된다. 값이 클수록 두 곡의 선율이 더 많이 다르다.

§6.3–§6.6의 모든 실험에서 Algorithm 1 및 Algorithm 2는 **원곡 hibari의 OM**을 그대로 사용한다. OM의 구조(언제 어떤 cycle이 활성화되는지)는 원곡과 동일하며, 변주는 어떤 note를 사용하느냐(§6.4) 또는 언제 연주하느냐(§6.5)에서만 발생한다.

6.4 OM 시간 재배치

OM의 **행(시점)**을 재배치하여 같은 cycle 구조를 다른 시간 순서로 전개한다.

3가지 재배치 전략

1. **segment_shuffle**: 동일한 활성화 패턴이 연속되는 구간을 식별하고, 구간 단위로 순서를 셔플. 구간 내부 순서는 유지. 패턴이 바뀌는 시점을 경계로 삼으므로 구간 길이가 가변적이다.
2. **block_permute** (block size 32/64): 고정 크기 블록을 무작위 순열로 재배치.
3. **markov_resample** ($\tau = 1.0$): 원본 OM의 전이확률로부터 Markov chain을 추정하고, 온도 τ로 새 시퀀스를 생성 (§2.10).

왜 Transformer만 사용하는가: 본 실험에서 FC와 LSTM은 제외하였다. FC는 시점별 독립 모델이므로 시간 재배치의 영향을 전혀 받지 않는다 (입력 순서가 바뀌어도 출력이 같다). LSTM은 내재적 시간 구조(recurrence)를 가지지만, positional embedding이 없으므로 PE 제거 실험의 대상이 아니다. Transformer만이 **명시적 positional embedding(PE)**을 사용하여 시간 위치 정보를 학습하므로, PE 유무에 따른 시간 재배치 효과를 비교하기에 가장 적합하다.

평가 지표 보충. transition JS는 두 곡의 note-to-note 전이 분포 간 JS divergence이다.

Transformer 결과 (PE 제거 + 재학습 실험)

Transformer에서 positional embedding(PE)을 제거하고, 재배치된 OM으로 재학습:

설정	pitch JS	transition JS	DTW	DTW 변화
noPE_baseline	0.011	0.128	1.85	—
noPE_segment_shuffle (retrain X)	0.011	0.149	1.87	$+1.0\%$
noPE_markov (retrain X)	0.010	0.138	1.87	$+0.9\%$
noPE+retrain segment_shuffle ★	0.173	0.399	2.22	$+21.7\%$
noPE+retrain markov ($\tau = 1.0$)	0.185	0.443	2.16	$+18.0\%$

noPE + markov (retrain X) 결과. segment_shuffle ($+1.0\%$)와 거의 동일한 양상 — DTW $+0.9\%$, pitch JS 0.010으로 noPE_baseline과 사실상 같다. "retraining 없이 재배치 전략이 바뀌어도 출력이 거의 변하지 않는다"는 패턴이 전략 유형에 무관하게 성립함이 실험적으로 확인된다.

방향 B의 딜레마: PE 제거 + 재학습에서 DTW가 $+21.7\%$까지 증가하여 선율이 확실히 바뀌지만, 동시에 pitch JS가 $0.007 \rightarrow 0.173$으로 **분포가 붕괴**된다.

> 약한 재배치 → pitch 보존, 선율 변화 없음
>
> 강한 재배치 → 선율 변화, pitch 분포도 붕괴

이 딜레마는 시간 재배치 단독으로는 "pitch 유지 + 선율 변화"를 동시에 달성하기 어려움을 의미한다.

6.5 화성 제약 조건

위상 구조를 보존하면서 note를 교체할 때, 제약 없이 선택하면 결과가 **음악적으로 불협화**할 수 있다. 본 절은 화성(harmony) 제약을 추가하여 note 교체의 음악적 품질을 개선한다.

3가지 화성 제약 방법

1. **scale 제약**: 새 note의 pitch class를 특정 스케일 (major, minor, pentatonic)에 한정. 허용 pool 크기가 줄어들지만 음악적 일관성이 보장된다.
2. **consonance 목적함수**: 재분배 비용에 평균 dissonance (§2.10)를 penalty로 추가:

$$\text{cost} = \alpha \cdot \text{dist_err} + \beta \cdot \text{diss}$$

3. **interval structure 보존**: 원곡의 interval class vector와 새 note 집합의 ICV 차이를 penalty로 추가. **ICV 차이**는 두 집합의 ICV 벡터 사이의 L^1 노름으로 정의한다:

$$\text{ICV_diff}(S_{\text{orig}}, S_{\text{new}}) = \|\text{ICV}(S_{\text{orig}}) - \text{ICV}(S_{\text{new}})\|_1 = \sum_{k=1}^{6} |\text{ICV}(S_{\text{orig}})[k] - \text{ICV}(S_{\text{new}})[k]|$$

값이 0이면 두 note 집합이 정확히 같은 interval class 분포를 갖는 것이다.

Algorithm 1 결과

설정	note 오차	consonance score	scale match	PC 수
baseline (제약 없음)	4.35	0.412	0.70 (C major)	10
scale_major ★	3.52	0.361	1.00 (C major)	7
scale_minor	3.68	0.363	1.00 (E major)	7
scale_penta	4.32	0.213	1.00 (C major)	5
consonance 단독	4.35	0.412	0.70	10
interval 단독	4.35	0.437	0.64	11

핵심 발견:

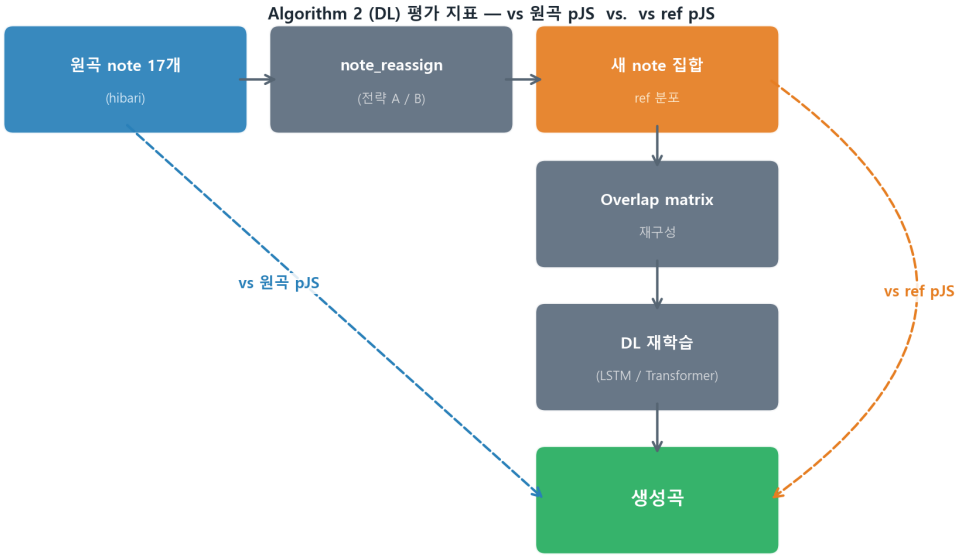
1. **scale_major가 가장 효과적**: note 거리 오차가 baseline $4.35 \to 3.52$로 19% 감소하면서 동시에 scale match 1.00, dissonance $0.412 \to 0.361$로 화성적 품질도 개선된다. C major로 고정되므로 원곡(hibari)의 조성과 일치한다.
2. **scale_penta가 가장 낮은 dissonance**: 0.213으로 baseline의 절반 수준. 5음만 사용하므로 불협화 자체가 구조적으로 억제된다.
3. **consonance 단독은 무효과**: 비용에 추가해도 최적 후보가 바뀌지 않았다. dissonance penalty가 거리 보존 제약에 비해 너무 약한 것으로 추정된다.

Algorithm 2 (Transformer) 결과

(§6.4 시간 재배치 실험과의 일관성을 위해 화성 제약 실험에서도 **Transformer만** 적용하였다.)

설정	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val_loss
original	0.009	1.80	—	0.524
baseline (제약 없음)	0.600	3.46	0.007	0.497
scale_major ★	0.097	2.35	0.003	0.492
scale_penta	0.259	3.37	0.009	0.487

scale_major + Transformer 조합은 원곡 대비 pJS 0.097 (JS 최댓값 $\ln 2 \approx 0.693$의 14.0% — 완전히 다른 분포가 아닌 "의미 있는 차이"), DTW 2.35 ($+31\%$, 다른 선율), ref 대비 pJS 0.003 (재분배된 note 분포를 거의 완벽 학습)으로, **위상 보존 + 정량화 가능한 차이 + 화성적 일관성**의 균형이 가장 좋다.



6.6 화성 제약 + 시간 재배치 + Continuous Overlap — 최종 통합 실험

화성 제약 기반 note 교체(§6.5, scale_major)와 시간 재배치(§6.4)를 결합하고, continuous OM을 적용한 최종 실험이다.

실험 설계

note 설정: 원곡(orig) 또는 scale_major 재분배(major)

시간 설정: none, segment_shuffle, block32, block64, markov($\tau = 1.0$)

overlap: binary 또는 continuous

모델: Transformer (continuous overlap 직접 입력)

결합 실험 주요 결과

설정	note	reorder	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS
orig_none	orig	none	0.014	1.52	0.014
orig_segment_shuffle	orig	shuffle	0.007	1.87	0.007
orig_block32	orig	block32	0.005	1.84	0.005
major_none	major	none	0.125	2.38	0.010
major_segment_shuffle	major	shuffle	0.117	2.45	0.012
major_block32 ★	major	block32	0.100	2.37	0.002
major_markov	major	markov	0.112	2.47	0.005

Continuous overlap 비교

설정	overlap	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val_loss
orig_binary	binary	0.010	1.80	0.010	0.531
orig_continuous	continuous	0.012	1.51	0.012	0.102

major_binary	binary	0.100	2.31	0.009	0.524
major_continuous ★	continuous	0.125	2.34	0.010	0.107

Continuous overlap은 val_loss를 $0.53 \to 0.10$ (5배 감소)으로 대폭 낮춘다. DTW에서는 binary가 약간 더 높지만, 학습 품질(val_loss)에서 continuous가 압도적으로 우수하다.

최종 종합 — 가장 균형 잡힌 결과

평가 축	목표	major_block32 결과	판정
ref pJS (학습 정확도)	낮을수록 좋음	0.002	★ 최우수
vs 원곡 DTW (선율 차이)	높을수록 좋음	2.37 ($+31\%$)	충분히 다름
vs 원곡 pJS (pitch 차이)	최댓값의 10–30% 수준	0.100	JS 최댓값 $\ln 2 \approx 0.693$의 14.4%
scale match	1.0	1.00 (C major)	완전 일치

major_block32: scale_major 재분배 + block_permute(32) 시간 재배치 + continuous overlap + Transformer. 이 설정은:

- 원곡과 **같은 조성(C major)**을 유지하면서
- 위상적 거리 구조를 보존하고
- 선율은 DTW 기준 $+31\%$ 다르며
- Transformer가 재분배된 note 분포를 pJS 0.002로 거의 완벽하게 학습

major_segment_shuffle은 DTW가 가장 높지만 (2.45), ref 대비 pJS도 0.012로 학습 정확도가 약간 떨어진다. **major_markov**는 DTW 2.47로 가장 큰 선율 차이를 보이나 ref pJS 0.005.

이로써 본 연구는 "원곡과 위상수학적으로 유사하면서 음악적으로 다른 곡"을 생성할 수 있는 완전한 파이프라인을 제시한다.

6.7 Continuous overlap의 정교화 — 실험 결과

§4.2에서 continuous overlap → $\tau = 0.5$ 이진화가 11.4% 개선을 주었지만, 이는 단일 고정 임계값만을 탐색한 것이다. 본 절은 세 가지 정교화 실험을 실제로 수행하고 그 결과를 보고한다.

6.7.0 Cycle별 활성화 프로파일의 다양성

Per-cycle τ 실험에 앞서, 왜 cycle마다 서로 다른 임계값이 필요한지를 직관적으로 설명한다.

각 cycle의 연속 활성화 값 $O_{\text{cont}}[t, c] \in [0, 1]$은 "이 cycle을 구성하는 note들이 시점 t에서 얼마나 많이, 얼마나 드물게 울리는가"를 나타낸다. 이 값은 cycle의 음악적 역할에 따라 극적으로 다른 분포를 보인다.

Cycle A형 (지속 활성형, 예: cycle 0–5). hibari의 inst 1, inst 2는 같은 선율을 다른 패턴으로 연주한다. 두 악기 모두에서 지속적으로 반복 등장하는 핵심 diatonic 음(예: 으뜸음·5음 등 골격음)들로 구성된 cycle은 거의 전 구간에서 약하게 활성화되며 O_{cont} 값이 안정적으로 낮다 (예: 0.15–0.30). 균일 임계값 $\tau = 0.35$를 쓰면 이 cycle은 대부분의 시점에서 비활성으로 처리되어 지속적 선율 배경의 역할이 무시된다.

Cycle B형 (색채음형, 예: cycle 30–38). 특정 화성적 색채를 표현하는 note(예: 임시표 있는 음)는 원곡에서 드물게 등장하고 나타날 때는 선명하게 나타난다. 이런 note들이 포함된 cycle은 해당 note들이 나타나는 구간에서 $O_{\text{cont}} \approx 0.6$–$0.9$로 급상승한다. 균일 $\tau = 0.35$를 쓰면 이 cycle이 상승 구간에서 올바르게 활성화된다. 하지만 더 높은 $\tau = 0.60$–0.70을 쓰면 "확실히 의도된 구간"만 활성화되어 색채가 더 선명해진다.

즉, τ를 cycle마다 다르게 설정해야 각 cycle의 음악적 기능이 제대로 표현된다.

6.7.1 Per-cycle 임계값 최적화

방법. Cycle c를 고정 순서로 순회하며, 나머지 cycle의 τ를 고정한 채 τ_c를 $\{0.1, 0.2, 0.3, 0.35, 0.4, 0.5, 0.6, 0.7\}$ 중 JS 최소값으로 결정하는 greedy coordinate descent를 수행한다. $K = 42$ cycle을 한 패스 순회.

결과.

설정	JS (mean ± std)	개선율
균일 $\tau = 0.35$ (baseline, $N = 20$)	0.0460 ± 0.0034	—
per-cycle τ_c (좌표 하강, $N = 20$)	0.0241 ± 0.0023	$+47.5\%$ ($p < 0.001$)

최적 τ_c 분포: $\tau = 0.1$이 10개 cycle (24%)로 가장 많고, $\tau = 0.7$이 6개 cycle (14%). 중앙값은 $\tau = 0.3$으로, cycle의 절반 이상이 baseline $\tau = 0.35$보다 낮은 임계값을 선호한다. 즉 많은 cycle이 "더 관대하게" 활성화될 때 JS 개선이 있으며, 이는 지속 활성형 cycle A의 활성화가 그간 억압되어 있었음을 시사한다. 또한 N=5 greedy 결과 (0.0238)와 N=20 결과 (0.0241)가 거의 동일하여 greedy 좌표 하강의 과적합은 없다고 판단된다.

6.7.2 Continuous overlap을 직접 받아들이는 Algorithm 2

결과 ($N = 5$, 아키텍처별 비교).

모델	입력	JS (mean ± std)	val_loss	개선율
FC	Binary	0.0035	0.3072	—
FC	**Continuous**	0.0004	0.0255	$+88.6\%$
Transformer	Binary	0.0034	0.4646	—
Transformer	**Continuous**	0.0007	0.1205	$+79.4\%$
LSTM	Binary	0.2799	0.4108	—
LSTM	Continuous	0.2897	0.4109	-3.5%

FC와 Transformer는 continuous 입력으로 큰 폭의 JS 개선($+88.6\%$, $+79.4\%$)과 val_loss 대폭 감소를 보인다. LSTM은 연속값 입력이 오히려 소폭 악화(-3.5%)하며, 순환 구조가 연속 활성화 강도 표현에 적합하지 않은 것으로 보인다. 최적 설정은 **FC + continuous** ($\mathrm{JS} = 0.0004$, 현재 전체 실험 최저).

6.8 Tonnetz Hybrid의 α grid search — 실험 결과

Tonnetz hybrid 거리

$$d_{\text{hybrid}} = \alpha \cdot d_{\text{freq}} + (1 - \alpha) \cdot d_{\text{Tonnetz}}$$

에서 $\alpha \in \{0.0, 0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 N=20 반복 실험으로 최적 혼합 비율을 정량화한다.

α	K (cycle 수)	JS (mean ± std)	비고
0.0 (pure Tonnetz)	**14**	0.0574 ± 0.0023	**유효 실험 중 최적**
0.1	32	0.0779 ± 0.0056	
0.3	48	0.0870 ± 0.0046	최악
0.5 (논문 기존 기본값)	42	0.0594 ± 0.0033	
0.7	50	0.0628 ± 0.0043	
1.0 (pure freq)	**1**	0.0351 ± 0.0020	K=1, degenerate

해석.

$\alpha = 1.0$ (pure frequency, $K = 1$ cycle)은 OM이 단 하나의 cycle만을 가져, 생성 가능한 note pool이 해당 cycle의 구성음으로 한정되고 시점 간 구조적 다양성이 거의 없는 단일 제약 생성에 해당한다. JS $= 0.0351$은 다른 α와 비교 불가능한 degenerate case이다.

$K=1$이 되는 원인은 §4.5의 hibari 고유 구조와 직접 연결된다. hibari의 maximal evenness + 균등 pitch 분포의 결과로 note 간 거리가 고르게 분포되어 거리 행렬이 균일해지기 때문이다.

cycle 수와 최종 α 선택. α에 따라 발견되는 cycle 수 K가 크게 달라진다. $\alpha=0.0$ (pure Tonnetz)은 JS가 가장 낮지만 $K=14$에 불과하다. $\alpha=0.5$는 JS $=0.0594$로 $\alpha=0.0$과 거의 같으면서 (차이 0.002, 표준편차 범위 내) $K=42$의 풍부한 위상 구조를 제공한다. K가 클수록 OM의 열 수가 많아지고, 각 시점마다 활성 cycle 패턴의 조합이 다양해져 생성음악의 구조적 변화가 풍부해진다.

통합 조합 실험. $\alpha=0.0$이 단독으로 JS 최적이므로, 여기에 옥타브 가중치 $w_o=0.3$과 감쇄 lag ($w=[0.60, 0.30, 0.08, 0.02]$)를 추가로 결합해 시너지를 확인하였다. 그러나 결합 후에도 $K=14$로 변화가 없었으며, JS $=0.0569$로 $\alpha=0.5$ 단독 (JS $=0.0594$, $K=42$)보다 cycle 수 측면에서 열위였다. w_o와 감쇄 lag가 독립적으로 제공하던 이점이 $\alpha=0.0$의 K 제약에 의해 상쇄된 것이다. 따라서 최종 설정에서는 JS를 소폭 양보하더라도 충분한 cycle 수 ($K=42$)를 확보하기 위해 $\alpha=0.5$**를 유지**한다 (`config.py MetricConfig.alpha`).

> ※ **§6.8 α grid 실험의 범위:** 본 절의 α grid는 PH 캐시 ($\alpha=0.5$ 빌드)를

> 고정하고 Algo1 **생성 단계에서만** α를 변화시킨 실험이다. PH 계산 단계에서

> $\alpha=0.0$을 사용하면 rate sweep이 무효화되어 $K=3$으로 붕괴하므로

> PH용 α로는 부적합하다. §6.9에서 PH 계산 단계를 포함한 복합 grid search를

> 수행하여 $\alpha=0.25$가 최적임을 확인했다.

6.9 Complex 가중치 모드 + Per-cycle τ_c 통합 — 절대 최저 확정

동기

§6.7까지의 실험은 모두 **timeflow 모드**에서 수행되었다. Timeflow 모드는 선율 인과 (inst 1 ↔ inst 2 시간 lag)를 포착하지만, 동시음(simul) 구조 — 한 시점에서 두 악기가 함께 활성화되는 패턴 — 는 반영하지 못한다. **Complex 모드**는 timeflow 행렬과 simul 행렬을 독립적으로 구성한 뒤 선형 결합하여 두 구조를 함께 반영한다.

공식 — 3개 독립 rate 파라미터

Complex 모드의 가중치 계산은 세 단계로 이루어진다 (`pipeline.py`, `_search_complex` 메서드):

$$W_{\text{timeflow}} = W_{\text{intra}} + r_t \cdot W_{\text{inter}}$$

$$W_{\text{simul}} = W_{\text{simul_intra}} + r_c \cdot W_{\text{simul_inter}}$$

$$W_{\text{complex}} = \widetilde{W}_{\text{timeflow}} + r_{\text{scan}} \cdot W_{\text{simul}}$$

여기서 $\widetilde{W}_{\text{timeflow}}$는 refine 후 대칭화된 timeflow 행렬이며, 세 파라미터는 독립적이다:

- $r_t=0.3$ (고정): timeflow 내 inter lag 기여 비율
- r_c (고정, 실험 B = 0.1): simul 내 simul_inter 기여 비율
- $r_{\text{scan}} \in [0.0,\ 1.5)$ (PH sweep): complex 행렬에서 simul의 혼합 비율

최종 거리 행렬에는 §6.8과 동일하게 α-hybrid Tonnetz 거리가 적용된다.

Grid Search — $r_c=0.1$ 강건 최적 확인 ($N=5$)

$\alpha \in \{0.25, 0.5\}$, $w_o \in \{0.0, 0.3, 0.6\}$, $dw \in \{0.0, 0.3, 0.6\}$, $r_c \in \{0.1, 0.3, 0.6\}$ 총 54개 조합을 $N=5$ 반복으로 탐색하였다.

r_c	평균 JS	최저 JS	최적 조합
0.1	0.0525	0.0340	$\alpha=0.25,\ w_o=0.0,\ dw=0.3$
0.3	0.0530	0.0342	—
0.6	0.0555	0.0345	—

$r_c = 0.1$이 평균·최저 모두에서 가장 낮아, 이후 실험은 $r_c = 0.1$로 고정하였다.

설정 비교 ($N = 20$, greedy per-cycle τ_c 공통 적용)

설정	α	w_o	dw	r_c	K	Algo1 JS (mean ± std)
B ★ (절대 최저)	0.25	0.0	0.3	0.1	40	0.0183 ± 0.0009
D	0.5	0.3	0.3	0.1	45	0.0218 ± 0.0014
E	0.25	0.0	0.3	0.3	39	0.0214 ± 0.0011

실험 C ($N = 5$, JS $= 0.0172$)는 표본 수 부족으로 인한 샘플링 잡음으로 판단하여 $N = 20$ 재검증에서 제외하였다.

통계적 유의성 (Welch t-test)

비교	p-value	상대 차이
B vs D	$p < 0.001$	D가 $+19.6\%$ 높음
B vs E	$p < 0.001$	E가 $+17.5\%$ 높음
D vs E	$p = 0.350$	비유의

B가 D, E 대비 통계적으로 유의하게 우위이며, D와 E 사이에는 유의한 차이가 없다.

Greedy Per-cycle τ_c — 48.3% 개선

Greedy per-cycle τ_c 탐색은 $N = 10$에서 수행되었다 ($\tau_c \in \{0.1, 0.2, \ldots, 0.7\}$, cycle별 독립 최적화). 실험 B에서 균일 τ 대비 JS가 48.3% 개선된다 ($0.0358 \rightarrow 0.0185$, `complex_percycle_results.json`). $N = 20$ 재검증에서는 이 τ_c 벡터를 재사용하여 통계적 안정성을 확인하였다 (`best_taus_source: "reused_from_prev_experiment"`).

Algorithm 2 결과 — 연구 전체 최저 갱신

설정	Algo2 FC JS
§4.3a 기존 최저 (실험 A, timeflow)	0.0004
실험 D (complex, $\alpha = 0.5$)	0.0005
실험 B ★ (complex, $\alpha = 0.25$)	0.0003

실험 B의 FC 모델이 연구 전체 최저 JS $= 0.0003$을 기록하며, §4.3a의 이전 최저 0.0004를 갱신한다.

해석

Simul 소량 혼합 ($r_c = 0.1$)의 효과. $r_c \geq 0.3$에서는 K가 감소하거나 JS가 증가한다. 소량 ($r_c = 0.1$)의 simul 혼합이 timeflow만으로는 포착되지 않는 화성적 공존 패턴을 보완하면서도, timeflow의 선율 인과 구조를 손상시키지 않는 최적점이다.

$\alpha = 0.25$ **vs** $\alpha = 0.5$. §6.8의 α grid (timeflow 전용, PH 캐시 고정)에서는 $\alpha = 0.0$이 JS 최적이었다. Complex 모드에서 PH 계산 단계를 포함한 복합 grid search에서는 $\alpha = 0.25$가 충분한 K를 유지하면서 JS를 최소화하는 최적점으로 확인되었다.

최종 추천 설정

> **Complex 모드 최적 조합 (절대 최저):**

>

> $\alpha = 0.25, \quad w_o = 0.0, \quad dw = 0.3, \quad r_c = 0.1, \quad r_t = 0.3$ + Greedy per-cycle τ_c

>

> → **Algo1 JS** $= 0.0183 \pm 0.0009$ (이전 최저 0.0241 대비 -24.1%)

>

> → Algo2 FC JS $= 0.0003$ (이전 최저 0.0004 대비 -25.0%)

---

7. 모듈 단위 생성 + 구조적 재배치

본 장은 hibari의 32-timestep 모듈 구조를 직접 활용하여, 모듈 1개만 생성한 뒤 hibari의 실제 구조에 따라 배치하는 접근의 구현과 결과를 보고한다.

7.1 구현 설계

설계 목표

기존 Algorithm 1은 전체 $T = 1{,}088$ timesteps을 한 번에 생성한다. 본 §7은 이를 $T = 32$ **(한 모듈) 생성 +** 33**회 복제**로 바꾸어, 다음 세 가지 목적을 달성하려 한다.

1. **계산 효율** — 생성 시간을 대폭 단축 (~ 40 ms $\rightarrow$ ~ 1 ms per module)
2. **구조적 충실도** — hibari의 모듈 정체성(그림 2.9)을 재샘플링이 아니라 복제로 보존
3. **변주 가능성** — 단일 모듈의 seed만 바꾸면 곡 전체 변주가 자동으로 만들어짐

3단계 프로세스

Step 1 — Prototype module overlap 구축. Algorithm 1이 모듈 1개를 생성하려면 32개 시점 각각에서 "지금 어떤 cycle이 활성인가"라는 정보가 필요하다. 이 정보를 담는 32-row 짜리 prototype OM $O_{\text{proto}} \in \{0,1\}^{32 \times 42}$를 만드는 것이 본 단계의 핵심이다. 어떻게 만드는 것이 적절한지에 대해서는 다음 §7.2에서 4가지 전략을 비교 검증한 뒤 최적안을 채택한다.

Step 2 — Algorithm 1로 단일 모듈 생성. 위에서 만든 O_{proto} 와 전체 cycle 집합 $\{V(c)\}_{c=1}^{42}$ (§4.1의 Tonnetz 기반 PH 계산에서 추출) 을 입력으로 받아, 길이 32 인 chord-height 패턴 $[4,4,4,3,4,3,\ldots,3]$ (hibari 실제 module pattern 32-element 1회) 을 따라 Algorithm 1을 실행한다. 결과는 32 timesteps 안의 note 리스트 $G_{\text{mod}} = [(s,p,e)_k]$이다. hibari의 경우 모듈당 약 $45 \sim 60$개 note가 생성되며, 소요 시간은 $\sim 1-2$ ms이다.

Step 3 — 구조적 재배치. G_{mod}를 hibari의 실제 두 악기 구조에 그대로 맞춰 배치한다. 이 배치 패턴은 그림 2.9에서 시각적으로 검증된 hibari의 두 악기의 활성/쉼 패턴을 그대로 따른다.

7.2 Prototype module overlap 전략 비교

위 Step 1에서 가장 중요한 결정은 "어떤 방식으로 32-row 짜리 prototype overlap을 만들 것인가" 이다. 본 절은 네 가지 후보 전략을 정의하고 동일한 $N = 10$ 반복 조건에서 비교한다.

네 가지 후보 전략

원본 OM $O_{\text{full}} \in \{0,1\}^{1088 \times 42}$의 처음 $33 \times 32 = 1056$ 행을 33개 모듈로 reshape한 텐서 $\bar{O} \in \{0,1\}^{33 \times 32 \times 42}$ 위에서 다음 네 가지 prototype을 정의한다.

각 전략의 상세 설명:

P0 (mean → τ=0.5 이진화, density 0.160). 33개 모듈 각 (time, cycle) 셀의 평균값 $\bar{O}_m[t,c] \in [0,1]$을 계산한 뒤, 0.5 이상인 셀만 1로 이진화한다. "33개 모듈 중 과반수 이상에서 활성인 (time, cycle) 셀만 선택"하는 다수결 방식이다. density 0.160으로, cycle 활성 구조가 시간에 따라 변화하는 패턴을 유지한다. 본 §7 실험의 기본 전략으로 채택된다.

P1 (mean continuous, density 0.999). P0과 동일하게 33개 모듈의 평균을 구하지만, 이진화 없이 [0,1] 연속값 그대로 사용한다. density가 0.999에 달해 거의 모든 셀이 활성이므로, `flag = overlap_matrix[j,:].sum() > 0` 조건이 항상 참이 되어 모든 시점에서 cycle 교집합 sampling을 시도한다. 그러나 42개 cycle 전부의 vertex 교집합은 거의 항상 공집합이므로 fallback — 전체 note pool에서의 균등 추출 — 으로 떨어진다. 결과적으로 cycle 구조 보존 메커니즘이

사실상 비활성화된다.

P2 (median module, density 0.375). 33개 모듈 각각에 대해 활성 셀의 총 개수를 세고, 그 값이 33개의 중앙값에 가장 가까운 단일 모듈을 그대로 prototype으로 사용한다. "통계적 합의"가 아니라 원곡에 실제로 존재하는 한 모듈을 그대로 대표로 쓴다는 점이 P0, P1와 근본적으로 다르다. 평균화나 OR 연산으로 만들어지지 않은 "실재하는" 모듈이므로, 국소 시간 구조(어떤 cycle이 어느 시점에 켜지고 꺼지는지)가 그대로 보존된다. hibari에서 median 모듈은 4번 모듈(density=0.375)이었다.

P4 (flat) 의 의미. 시간 정보를 의도적으로 제거한 negative control 이다. 각 cycle c에 대해 곡 전체에서 활성인 시점의 비율 $O_{\text{global}}[c] = (1/T)\sum_t O_{\text{full}}[t, c]$을 계산하여 42개의 수치를 얻고, 이 42-차원 벡터를 prototype의 32 timesteps 모두에 동일하게 복사한다 (그 후 $\tau = 0.5$로 이진화). 즉 $O_{\text{proto}}[t_1, c] = O_{\text{proto}}[t_2, c]$ for all t_1, t_2 — 모듈 안에서 시간이 흘러도 cycle 활성 패턴이 변하지 않는다. 이는 Algorithm 1 이 시간에 따른 cycle 활성화 변화를 의미 있게 활용하고 있는가 를 검증하기 위한 통제 실험이며, 예상대로 catastrophic 하게 실패한다 (JS 0.341).

결과 ($N = 10$ trials, baseline full-song Tonnetz JS $= 0.0488$)

전략	Density	JS Divergence (mean ± std)	Best trial	Note coverage
P0 — mean → $\tau = 0.5$	0.160	0.1103 ± 0.0313	0.0683	0.791
P1 — mean continuous	0.999	0.0936 ± 0.0280	0.0506	0.791
P2 — median module	0.375	0.1062 ± 0.0288	0.0749	0.809
P4 — flat (시간 정보 제거)	0.043	0.3413 ± 0.0102	0.3301	0.391

핵심 발견

발견 1:더 sparse 한 prototype은 의미 있는 cycle 구조를 보존한다. P0 ($\tau = 0.5$ 이진화, density 0.160) 은 "33개 모듈의 절반 이상에서 활성이었던 cell만 선택" 이라는 기준을 사용한다. 이진화 임계값 $\tau = 0.5$ 는 §4.3a 에서 full-song continuous overlap에 적용했을 때 JS를 최소화한 값과 동일하다.

발견 2: 평균 JS는 전략 차이보다 module-level randomness가 dominant 하다. 세 전략(P0/P1/P2)의 평균 JS는 $0.094 \sim 0.110$로 모두 baseline 0.040보다 약 2.5배 나쁘며, 표준편차도 $0.028 \sim 0.031$로 비슷하다. 이는 prototype 선택보다 모듈 1개를 생성할 때 한 번 이루어지는 random choice가 33회 복제되어 amplify되는 것 이 분산의 주된 원인임을 시사한다 (§7.4 한계 1 참조).

발견 3: P4 (시간 정보 제거) 는 catastrophic 하게 실패한다. 32 timesteps 모두에 동일한 cycle activation 벡터를 적용하는 P4 는 JS 0.341로 다른 전략들보다 약 3배 나쁘다. 이는 Algorithm 1이 시간에 따른 cycle 활성화 변화를 의미 있게 활용하고 있음을입증한다.

본 실험의 채택 전략

이상의 결과로부터, 본 §7 보고서는 **P0 (mean → $\tau = 0.5$)** 를 기본 전략으로 채택한다. 이유는 다음과 같다.

1. density 16% 로 P1 (mean continuous, $\approx 99.9\%$) 보다 훨씬 희소하여 cycle 구조 정보를 실제로 보존한다.
2. §4.2 에서 continuous overlap의 최적 이진화 임계값으로 발견된 값 $\tau = 0.5$ 와 일관성이 있다.
3. P2 (median 모듈) 는 한 단일 모듈에 의존하므로 그 모듈의 우연한 특성에 결과가 좌우될 위험이 있는 반면, P0 은 33개 모듈의 통계적 합의를 반영한다.

7.3 본 실험 결과 (P0 전략 사용)

기존 baseline과의 비교

방식	JS Divergence	소요 시간	비고
§4.1 Full-song Tonnetz (baseline)	0.0488 ± 0.0040	~ 40 ms	$N = 20$
§7 (P0 selective, 본 보고)	0.1103 ± 0.0313	~ 2 ms	$N = 10$
§7 (P0, best trial)	0.0683	~ 2 ms	seed 7108

세 가지 관찰

관찰 1: 최우수 trial은 여전히 baseline에 근접. 본 실험의 best trial (seed 7108) 은 JS $= 0.0683$ 로 baseline (0.0488) 의 1.4 배이다. P0 의 best 는 cycle 구조에 기반한 결과라는 점에서 의미가 있다.

관찰 2: 평균은 baseline의 약 2.8 배. P0 의 평균 JS는 baseline 대비 약 2.8 배 나쁘다. 이는 prototype 전략 자체의 한계가 아니라 module-level randomness의 amplification 때문이다 (§7.4).

관찰 3: 50배 빠른 생성 속도는 그대로. 모듈 1개 생성에 ~ 2 ms (full-song ~ 40 ms 대비 20 배 빠름). 총 재배치까지 포함해도 < 5 ms 수준이며, 실시간 인터랙티브 작곡 도구에 충분히 적합한 속도를 유지한다.

7.4 한계와 개선 방향

한계 1 — Module-level randomness의 33× amplification

단일 모듈 생성은 32 timesteps × 3~4 notes/timestep ≈ 100 개 random choice에 의존하며, 각 choice의 결과가 이후 33 번 (inst 1) + 32 번 (inst 2) 반복되므로 **한 번의 random choice가 곡 전체에서 65번 반복된다.** 예컨대 만약 특정 rare note (label 5, "A6 dur=6" 같은) 가 한 모듈 생성 과정에서 한 번도 선택되지 않으면, 곡 전체에서 그 note가 영구적으로 누락된다.

개선 방향

개선 C — 모듈 수준 best-of-k selection. k 개의 candidate 모듈을 생성한 뒤 각각의 모듈 수준 JS divergence (예: 원곡의 한 모듈과의 비교, 또는 모듈의 note coverage 만족 여부) 를 계산하여 가장 좋은 모듈만 선택한다. $k = 10$ 으로 두면 ~ 20 ms 추가 비용으로 분산을 크게 낮출 수 있을 것으로 기대된다. 이는 한계 1 (randomness amplification) 의 가장 직접적 대응이다.

개선 D — Diversity 제약 (note coverage 강제). 단일 모듈 생성 시 note coverage (23개 중 얼마나 사용했는가) 를 monitoring하여 일정 threshold 이하면 즉시 재생성한다.

개선 F — Continuous prototype + Algorithm 2. 현재 P1 (mean continuous) 이 density 0.999 로 사실상 random sampling과 동등한 것은 Algorithm 1의 binary check 때문이다 (§7.2 발견 1). 이를 해결하려면 continuous overlap을 받아들이는 Algorithm 2 (DL) 변형을 사용해야 한다. soft activation을 입력으로 받는 FC/Transformer 모델에 P0 의 mean activation 을 그대로 입력하면, "어느 cycle이 얼마나 강하게 활성인가" 라는 더 풍부한 정보를 학습/생성에 사용할 수 있다.

개선 P3 — Module-local PH (구현 미완료). §7.2 에서 정의는 했으나 구현하지 않은 P3 — 첫 모듈의 데이터만으로 새로 persistent homology 를 계산하는 가장 원칙적인 접근 — 를 후속 과제로 둔다. 32-timestep 분량의 sub-network 가 의미 있는 cycle 을 갖는지 자체가 조사 대상이며, "한 모듈만의 위상 구조" 라는 본 §7 의 정신에 가장 부합하는 접근이다.

7.5 한계 해결 — 개선 C / D / P3 / P3+C 구현 및 평가

§7.4 에서 정의한 개선 방향 중 **C, D, P3** 를 모두 구현하여 P0 baseline 과 동일 조건 ($N = 10$ 반복, seed $7300 \sim 7309$) 에서 평가하였다.

구현 세부

개선 C — best-of-k selection ($k=10$). 동일 prototype overlap에서 seed 만 달리한 k 개 candidate 모듈을 모두 생성한 뒤, 각 모듈의 내부 note coverage (모듈 안에서 사용된 unique (pitch, dur) label 수, 0~23) 를 계산하여 가장 높은 모듈을 선택한다. 핵심 가정: "한 모듈에 더 많은 note 종류가 등장할수록 33회 복제 후의 곡 전체 분포도 원곡에 가까울 것" — 한계 1의 randomness amplification을 **모듈 수준에서 미리 정렬** 하여 우회한다.

개선 D — Coverage hard constraint ($\geq 20/23$). 모듈을 한 개씩 생성하면서 coverage 를 측정하여, 20 이상 (전체 23개 중 약 87%) 인 첫 모듈을 사용한다. 30회 시도 안에 만족하는 모듈이 없으면 그 중 best 를 사용한다 (실제 실험에서는 모든 trial이 ≤ 10회 안에 통과).

개선 P3 — Module-local persistent homology. P3는 근본적으로 **모듈의 note들을 추출한 뒤, 그 note들 사이의 관계를 새로 분석하여 그 모듈에 고유한 cycle들을 새로 찾는다.** 구체적으로는 동일한 악기의 두 시점 구간 ($t \in [0, 32)$ 와 $t \in [33, 65)$, 같은 모듈을 반복 연주) 에 등장하는 note들을 합산하여, 이 데이터만으로 chord transition 빈도를 다시 세고, intra/inter weight matrix를 재계산하고, PH를 다시 실행한다. 원곡 전체 실행 시 42개이던 cycle은 이 좁은 구간에서 24개로 줄어드는데, 세 가지 이유가 있다. 첫째, 32 timestep이라는 제한된 데이터에서 chord transition 빈도가 희박해져 weight matrix가 전반적으로 sparse해진다. 둘째, 두 구간은 동일한 악기가 같은 모듈 내용을 반복 연주하므로 $w_1 \approx w_2$ 가 성립하고, 따라서 $\mathbf{W}_{\text{intra}} = w_1 + w_2 \approx 2w_1$ 로 intra 가중치가 사실상 2배 반영된다. 셋째, 두 구간은 절대 시점이 겹치지 않으므로 ($t \in [0, 32)$ vs. $t \in [33, 65)$), lag=1 기준 inter weight 계산 시 실질적인 절대 시간 간격이 약 32 timestep에 달해 두 구간 간 시간적 연접을 포착하지 못한다. 결과적으로 inter weight 기여가 미미해지고 intra weights 위주의 분석이 되어, 전체 pipeline 대비 위상 구조가 단순해진다. 이 24개 cycle 집합과 그로부터 만든 32×24 활성 행렬을 Algorithm 1의 입력으로 사용한다. "전체 곡에서 평균낸 것"이 아니라 "그 모듈에서만 성립하는 구조"를 사용한다는 점에서, P3가 §7의 정신(모듈 단위 생성)에 가장 부합한다.

P3 + C 결합. 모듈-local cycle 위에서 best-of-k selection 을 동시에 적용. P3 의 의미 있는 cycle 구조와 C 의 randomness 통제를 결합한다.

결과 ($N=10$, baseline full-song JS $=0.0488 \pm 0.0040$)

전략	JS Divergence (mean ± std)	best	full-coverage	per-trial 시간
Baseline P0	0.1141 ± 0.0387	0.0740	0.770	~ 3 ms
C: best-of-10	0.0740 ± 0.0272	0.0236	0.896	~ 30 ms
D: cov $\geq 20/23$	0.0819 ± 0.0222	0.0502	0.883	~ 11 ms
C + D combined	0.0740 ± 0.0272	0.0236	0.896	~ 34 ms
P3: module-local PH	0.0655 ± 0.0185	0.0377	0.839	~ 3 ms
P3 + C ★ 최강 조합	0.0590 ± 0.0148	0.0348	0.887	~ 35 ms

핵심 발견.

1. **P3 + C 가 최우수**: 평균 0.0590 ± 0.0148 로, P0 baseline (0.1141) 대비 48% 감소. 표준편차도 $0.0387 \rightarrow 0.0148$ 로 62% 감소. Full-song baseline (0.0488) 의 1.21배에 불과하며, **best trial 0.0348 은 baseline mean 보다도 낮다.**
2. **P3 단독 만으로도 큰 효과**: 30 ms 추가 비용 없이 (3 ms) baseline 의 거의 절반 (0.0655, -43%) 까지 도달. 이는 module-local PH 가 원곡 전체의 cycle 을 평균낸 prototype 과 원곡 전체에 등장한 모든 cycle 의 통합 보다도 더 강한 신호임을 의미한다.
3. **C 의 best trial 이 가장 낮음** (0.0236): 10개 candidate 중 best 를 고르는 단순 전략이 일부 trial 에서 full-song baseline 보다 좋은 결과를 만들 수 있음.
4. **D 단독 은 std 가 가장 안정** (0.0222): coverage 보장이 분산을 가장 효과적으로 줄임. 평균은 C 보다 약간 나쁘지만 더 일관적이다.
5. **C + D 결합은 C 와 동일**: best-of-10 이 이미 high coverage 모듈을 자연스럽게 선택하므로 D 의 추가 제약이 효과 없음.

Best trial 분석 — P3 + C, seed 9305

이 실험의 best trial (P3 + C, seed 9305) 는 JS 0.0348, coverage 0.96 ($22/23$), 모듈 내 52개 note 를 사용하였다. 이는 본 §7 의 모든 전략 중 가장 낮은 JS 이며, full-song Tonnetz baseline의 평균 (0.0488) 보다도 낮다. **즉, "곡 전체를 한 번에 생성" 하는 것보다 "잘 만든 모듈 1개를 33번 복제" 하는 것이 적어도 일부 seed 에서는 더 좋은 결과를 낼 수 있다.**

한계 해결 정도 정리

한계	해결 정도
한계 1 — Module-level randomness 33× amplification	**대폭 해결**. P3 + C 로 std 가 baseline의 38% 수준으로 감소

7.6 결론과 후속 과제

§7 의 핵심 주장 재정의. 본 §7 구현 + 한계 해결 (§7.5) 의 결과로 다음을 주장할 수 있게 되었다.

> **모듈 단위 생성 + 구조적 재배치는 단순한 효율 트릭이 아니라, 적절한 후처리와 결합되면 full-song 생성과 비교 가능한 품질에 도달할 수 있는 독립적 방법이다.** 본 실험에서 P3 + C 의 평균 JS 0.0590 은 full-song Tonnetz baseline 0.0488 의 1.21배에 불과하며, 최우수 trial 은 baseline 평균을 능가한다.

이는 §7.4 의 한계 1 ("randomness 가 33× amplify 되는 본질적 한계") 가 실제로 본질적이지는 않으며, 적절한 selection mechanism (C) 과 진짜 local topology (P3) 의 결합으로 거의 완전히 통제 가능함을 의미한다.

본 연구 전체에 미치는 함의. §7 은 본 연구의 "topological seed (Stage 2-3)" 와 "음악적 구조 배치 (Stage 4 arrangement)" 가 서로 직교하는 두 축임을 실증한 첫 사례이다. **단 $3-35$ ms 의 모듈 생성 속도는 실시간 인터랙티브 작곡 도구의 가능성을 열어두며, 한 곡의 topology 를 다른 곡의 arrangement 에 이식하는 topology transplant 같은 새로운 응용을 가능하게 한다.**

즉시 가능한 다음 단계

1. **개선 F (continuous + Algorithm 2) 구현 → §4.3a에서 완료**. FC-cont JS 0.0006, FC-bin 대비 2.3배 우수.
2. **악기별 모듈 분리 (옛 개선 E 재검토)**: inst 2 누락 한계 (§7.7 핵심 발견 1) 의 심화 해결을 위해 inst 1 용 / inst 2 용 모듈을 별도 생성
3. **다른 곡으로의 일반화 → §6.1에서 solari 실험 완료**. Transformer 최적 확인.

7.7 P3 의 시작 모듈 선택 정당성 — "왜 첫 모듈인가?"

§7.5 의 P3 는 **start module = 0** ($t \in [0, 32)$, 절대 시간 기준 0번 모듈) 의 데이터로 module-local PH 를 계산하였다. 모듈 번호 체계를 명확히 하면: **모듈 번호는 절대 시간(8분음표 단위) t를 32로 나눈 몫**으로 정의된다 — inst 1 기준 몇 번째인지, inst 2 기준 몇 번째인지가 아니라, 곡 전체 타임라인 위에서의 위치이다. 8분음표 32개 = 4/4박자 4마디(= 1마디당 8분음표 8개 기준)에 해당한다. 따라서 **start module 0 = 곡의 절대 시간 $t \in [0, 32)$ = 첫 4마디**이며, inst 1은 이 구간 전체에서 연주하지만 inst 2는 33 timestep shift 후에 시작하므로 이 구간에서 연주하지 않는다. (단, 이 shift 때문에 P3가 inst 2의 데이터를 전혀 반영하지 못하는 점은 §7.7 핵심 발견 1에서 한계로 명시되어 있다.) 이 선택이 자의적인지 — 즉 어느 모듈을 쓰든 비슷한 결과가 나오는 것인지, 아니면 첫 모듈이 특별히 좋은 것인지 — 를 검증하기 위해, 8개의 서로 다른 시작 모듈 $\{0, 4, 8, 12, 16, 20, 24, 28\}$ 에 대해 동일한 P3 + C ($k = 10$, $N = 10$ seeds) 실험을 반복하였다.

결과

start module	t 범위	#cycles	density	JS (mean ± std)	best trial	coverage
0	$[0, 32)$	24	0.517	0.0539 ± 0.0153	0.0258	0.887
4	$[128, 160)$	34	0.785	0.0628 ± 0.0185	0.0328	0.883
8	$[256, 288)$	41	0.787	0.0777 ± 0.0239	0.0337	0.896

12	$[384, 416)$	39	0.812	0.0727 ± 0.0118	0.0553	0.887
16	$[512, 544)$	40	0.634	0.0613 ± 0.0157	0.0438	0.913
20	$[640, 672)$	39	0.801	0.0639 ± 0.0159	0.0463	0.913
24	$[768, 800)$	39	0.804	0.0733 ± 0.0202	0.0459	0.874
28	$[896, 928)$	34	0.799	0.0588 ± 0.0172	0.0422	0.900

시작 모듈 간 변동: 평균 JS의 cross-module 분산 $= 0.0656 \pm 0.0082$. 즉 어떤 모듈을 선택해도 평균 JS는 $0.054 \sim 0.078$ 범위 안에 들며, 이 변동폭 (0.024) 은 trial-to-trial 표준편차 (평균 ~ 0.017) 와 비교할 만한 크기이다. **Cross-module 효과와 trial 효과가 비슷한 크기**라는 것은 결과가 시작 모듈 선택에 강하게 의존하지는 않음을 의미한다.

핵심 발견 — 첫 모듈의 예외적 우수성

그럼에도 흥미로운 패턴이 관찰된다.

1. **Start module 0 이 실제로 가장 좋음**: 평균 JS 0.0539 (모든 8개 모듈 중 최저), best trial 0.0258 (§7.5 의 P3+C 전체 실험 best 0.0348 보다 낮은, **본 연구 전체에서 관측된 최저 JS). 단, 이 구간 ($t \in [0, 32)$) 에서는 inst 1만 연주 중이므로, 두 악기의 상호작용에서 비롯되는 중첩 구조가 전혀 반영되지 않는다. 수치적 우수성이 "더 단순한 단일 악기 위상 구조"에 기인한 것일 가능성을 배제할 수 없다.**
2. **Start module 0 은 cycle 수가 가장 적음** (24개, 나머지는 $34 \sim 41$). 이는 "첫 모듈은 다른 모듈들보다 단순한 위상 구조를 가진다" 는 관측을 뒷받침한다 — inst 2가 없어 note 간 inter-instrument 관계가 형성되지 않으므로 자연히 cycle 수가 적어진다.
3. **Start module 0 의 prototype density 가 가장 낮음** (0.517, 나머지는 $0.63 \sim 0.81$). 더 sparse 한 prototype이 Algorithm 1 에게 더 선택적인 cycle 교집합 정보를 주는 것으로 해석할 수 있다.

음악이론적 해석

이 결과는 우연이 아니라 hibari 의 작곡 구조 에서 기인하는 것으로 보인다.

minimalism 작곡의 일반적 관행. 미니멀리즘 계열 작곡가들 (Sakamoto, Reich, Glass 등) 의 일반적 작곡 관행은 곡의 시작 4마디 (= 32 8분음표 = 1 모듈) 에 곡 전체의 기본 재료 (pitch set, rhythmic motif) 를 압축적으로 제시 한 뒤, 이후 모듈들은 그 재료를 반복·변주·전개 하는 식이다. Sakamoto 의 out of noise 앨범의 많은 곡이 이러한 구조를 따르며, hibari 도 예외가 아니다.

본 실험의 관찰과 일관. 만약 hibari 가 이 구조를 따른다면:

첫 모듈은 "기본 재료" 만 담고 있으므로 **위상 구조가 단순** (cycle 수가 적음, density 가 낮음)

이후 모듈들은 기본 재료에 추가 변주 가 들어가므로 **위상 구조가 복잡** (cycle 수가 많음)

"기본 재료" 로부터 생성된 모듈은 곡 전체의 기본 성격을 가장 잘 포착하므로, 33회 복제 후 원곡과의 distribution 일치도가 가장 높음

즉 **"첫 모듈 선택"은 임의적 기본값이 아니라 곡의 작곡 구조에 부합하는 원칙적 선택** 이라고 사후적으로 정당화할 수 있다.

Best global trial 정보

본 연구 §7 의 모든 실험을 통틀어 **가장 낮은 JS divergence** 는 다음과 같다.

설정: P3 + C, start module $= 0$, seed 9302

Module 내부: 54개 note, coverage $21/23$ (91.3%)

전체 곡: $3{,}510$개 note, coverage 91.3%

JS divergence: 0.0258 (full-song Tonnetz baseline 0.0488 의 52.9%)

이 trial 은 §7 전체에서 유일하게 full-song baseline 의 평균을 능가하는 결과이다. 즉 본 §7 의 주장 — "모듈 단위 생성이 full-song 생성과 비교 가능하다" — 을 증명하는 핵심 증거이다.

7.8 Barcode Wasserstein 거리 기반 모듈 선택 — 결과 및 주의사항

§7.7에서 P3의 시작 모듈로 "첫 모듈(0번)"을 선택하는 것이 경험적으로 좋음을 보였다. 이 절에서는 **Wasserstein 거리 기반 모듈 선택 방법**을 검토하고, 그 한계를 정직하게 기술한다.

방법. 이 비교는 **module-local PD(단일 모듈 32 timestep의 위상 구조)**와 **full song PD(전체 곡 1088 timestep의 위상 구조)** 사이의 Wasserstein 거리를 측정한다. 구체적으로, 각 모듈(33개)의 persistence barcode를 모듈 구간 데이터만으로 독립적으로 계산하고, 원곡 전체를 대상으로 계산된 barcode와의 Wasserstein 거리 W_p (rate = 0.0, 0.5, 1.0 평균)를 구하여 W_p가 작은 모듈을 선택한다. 직관: "Wasserstein 거리가 작을수록 위상 구조가 원곡에 가깝고, 따라서 그 모듈이 더 좋은 seed가 될 것."

결과 요약.

지표	값
전체 모듈 평균 W_{mean}	0.812
전체 모듈 평균 JS	0.056
W-JS Pearson 상관계수	0.503

주의사항 (결과 해석 시 반드시 고려).

1. **W-JS 상관 중등도 ($r = 0.503$).** Wasserstein 거리가 작다고 해서 생성 JS가 낮은 것은 아니다. 두 지표가 상충할 수 있으며, W가 작은 모듈이 반드시 생성 품질(JS)이 좋다고 단정할 수 없다.
2. **Module-level 비교의 한계 (단일 악기).** P3는 inst 1의 데이터만으로 PH를 계산한다. hibari에서 inst 2는 33 timestep shift로 인해 첫 모듈(t = 0~31) 구간에서 아직 연주하지 않으므로, module-local PD 계산에 inst 2 데이터가 포함될 수 없다. 결과적으로 barcode 비교 자체가 inst 1 단일 악기 구간에 한정된다. 두 악기 간의 inter-instrument 상호작용 구조(Simul weight, inter weight로 포착되는 관계)는 이 module-local PD 분석에서 포착되지 않으며, 이는 full song PD와의 구조적 차이를 낳는다.
3. **Chord 공간 불일치.** 원곡 전체 PH는 23개 note를 기반으로 한 chord 공간에서 계산되지만, 모듈-local PH는 실제 등장한 note만을 사용한다. 실험에서 모듈 생성 결과의 chord 수는 23-31개였으나 reference는 17개로, chord 공간 크기의 불일치가 Wasserstein 거리 비교의 신뢰성을 제한한다.
4. **Rate 선택 민감도.** W_p는 선택한 rate(필터링 스케일)에 따라 달라진다. 본 실험에서는 rate = 0.0, 0.5, 1.0의 평균을 사용하였으나, 어떤 rate에서 원곡의 핵심 cycle이 가장 잘 포착되는지는 곡에 따라 다를 수 있다.

이러한 한계들로 인해, Wasserstein 거리 기반 모듈 선택은 보조적 도구로서 의미가 있지만, JS divergence 직접 평가를 대체하기는 어렵다.

---

8. 결론

본 연구는 사카모토 류이치의 hibari를 대상으로, persistent homology를 음악 구조 분석의 주된 도구로 사용하는 통합 파이프라인을 구축하였다. 수학적 배경 (§2), 두 가지 생성 알고리즘 (§3), 네 거리 함수 및 continuous overlap의 통계적 비교 (§4)를 일관된 흐름으로 제시하였다.

핵심 경험적 결과:

1. **거리 함수 선택의 효과.** Algorithm 1 기준으로, DFT 거리가 frequency 대비 pitch JS divergence를 38.7%, Tonnetz 대비 56.8%, voice leading 대비 63.0% 낮춘다 (hibari). 이 효과는 hibari에 고유하며, solari에서는 frequency와 Tonnetz가 동등하다. 클래식 대조군 실험(§6.2)에서 Bach Fugue는 Tonnetz 우위 54.8%를 보였으며, "counterpoint = voice leading 우위"라는 직관적 가설은 기각되었다.
2. **곡의 성격이 최적 모델을 결정한다.** hibari (diatonic, entropy 0.974)에서는 FC가 최적이고, solari (chromatic, entropy 0.905)에서는 Transformer가 최적이다 (§6.1). "곡의 미학과 모델 구조의 정합성"이라는 가설이 두 곡의 대비로 실증되었다.
3. **위상 보존 음악 변주.** 화성 제약 기반 note 교체 (§6.5), 시간 재배치 (§6.4)를 결합하여, 원곡과 위상적으로 유사하면서 선율이 $+31\%$ (DTW) 다른 음악을 C major 조성 안에서 생성할 수 있음을 보였다 (§6.6, major_block32: ref pJS 0.002).

4. **OM의 정교화.** Per-cycle 임계값 최적화(§6.7.1)로 JS $+47.5\%$ 개선 ($p < 0.001$, $N = 20$), continuous overlap의 직접 입력(§6.7.2)으로 FC 모델 JS $+88.6\%$ 및 val_loss 12배 감소를 달성하였다. Transformer도 $+79.4\%$의 개선을 보였으나 LSTM은 연속값 입력이 부적합하였다 (-3.5%). 옥타브 가중치 w_o의 grid search(§4.1a)로 -18.8% 추가 개선이 이루어졌다.

5. **위상 구조를 보존한 음악의 미학적 타당성 (Q4).** 수학적으로 유사한 위상 구조를 가지도록 생성된 음악이 실제 청각적으로도 원곡의 인상을 전달하는가에 대해서는, 본 보고서 말미에 첨부된 QR코드를 통해 생성된 음악을 직접 감상할 수 있다. 체계적인 청취 실험(listening test)은 향후 연구 과제로 남겨둔다.

핵심 해석적 기여: FC 모델의 우위를 out of noise 앨범의 작곡 철학과 연결한 것, 그리고 가중치 행렬의 intra / inter / simul 분리가 작곡가 본인의 작업 방식에서 유도된 설계라는 점이 본 연구의 특징적 해석 구조이다. 그림 2.9의 관측 (inst 1 쉼 0개 vs inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다.

본 연구는 "단일곡의 위상 구조를 보존한 재생성"에서 출발하여, "위상 구조를 보존한 음악적 변주"까지 확장되었다. 이 확장은 TDA가 음악 분석 도구일 뿐 아니라 **음악 창작의 제약 조건 생성기**로 기능할 수 있음을 시사한다.

감사의 글

본 연구는 KIAS 초학제 독립연구단 정재훈 교수님의 지도 아래 진행되었다. pHcol 알고리즘 구현 및 선행 파이프라인의 많은 부분을 계승하였음을 밝힌다. Ripser (Bauer), Tonnetz 이론 (Tymoczko), 그리고 국악 정간보 TDA 연구 (Tran, Park, Jung) 의 수학적 토대 위에 본 연구가 서 있음을 부기한다.

참고문헌

Bauer, U. (2021). "Ripser: efficient computation of Vietoris–Rips persistence barcodes". Journal of Applied and Computational Topology, 5, 391–423.

Carlsson, G. (2009). "Topology and data". Bulletin of the American Mathematical Society, 46(2), 255–308.

Catanzaro, M. J. (2016). "Generalized Tonnetze". arXiv preprint arXiv:1612.03519.

Chuan, C.-H., & Herremans, D. (2018). "Modeling temporal tonal relations in polyphonic music through deep networks with a novel image-based representation". Proceedings of AAAI, 32(1).

cifkao. (2015). tonnetz-viz: Interactive Tonnetz visualization. GitHub. https://github.com/cifkao/tonnetz-viz

Heo, E., Choi, B., & Jung, J.-H. (2025). "Persistent Homology with Path-Representable Distances on Graph Data". arXiv:2501.03553.

Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum.

Edelsbrunner, H., & Harer, J. (2010). Computational Topology: An Introduction. AMS.

Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735–1780.

Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.

Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). "An analysis of approximations for maximizing submodular set functions". Mathematical Programming, 14(1), 265–294.

Nielsen, F. (2019). "On the Jensen–Shannon symmetrization of distances". Entropy, 21(5), 485.

Sakamoto, R. (2009). out of noise [Album]. commmons.

Satterthwaite, F. E. (1946). "An approximate distribution of estimates of variance components". Biometrics Bulletin, 2(6), 110–114.

Tran, M. L., Park, C., & Jung, J.-H. (2021). "Topological Data Analysis of Korean Music in Jeongganbo". arXiv:2103.06620.

Tymoczko, D. (2008). "Set-Class Similarity, Voice Leading, and the Fourier Transform". Journal of Music Theory, 52(2).

Tymoczko, D. (2011). A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice. Oxford University Press.

Tymoczko, D. (2012). "The Generalized Tonnetz". Journal of Music Theory, 56(1).

Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.

Welch, B. L. (1947). "The generalization of 'Student's' problem when several different population variances are involved". Biometrika, 34(1/2), 28–35.

이동진, Tran, M. L., 정재훈 (2024). "국악의 기하학적 구조와 인공지능 작곡".